

2025级《程序设计基础课程设计》总结报告

学号	姓名	性别	班级	具体任务分工	所占比例	成绩
55250127	冯辉	男	01	项目组长，全面统筹 · 项目架构设计、编码规范制定与进度管理 · 主导药品管理模块（Drug CRUD / 库存预警 / 出入库） · 主导门诊发药流程（issuePrescription 六项校验 / 分精度计费 / 重复处方检测 / 保存顺序优化） · 主导 drug.c 全部功能开发与持久化 · 参与科室与床位管理模块（dept_bed.c）功能开发 · 负责系统集成测试、边界测试、容量测试 · 撰写五个专题分析报告及课程总结报告	35%	
55250129	聂溥成	男	01	系统架构师，基础层搭建 · 设计 his.h 全局头文件（8 个业务结构体 / 全部函数声明 / HIS_STRNCPY 宏） · 设计 his_config.h（长度常量 / 枚举 / 文件路径 / ID 前缀 / 业务默认值） · 实现 his_link.c 泛型单向链表（void* 深拷贝 / O(n) 查找 / 通用遍历打印） · 实现 his_tool.c 工具链（输入校验 / 原子写入 write-then-rename / 密码 nibble-swap / 身份证 GB11643 加权校验） · 参与科室与床位管理模块（dept_bed.c）功能开发 · 设计 his_main.c 三角色登录认证与会话变量管理	25%	
55250120	李腾飞	男	01	患者与医疗记录子系统 · 患者模块全功能（patient.c：CRUD / 姓名/手机号/身份证联动校验 / PIN 保护 / 自助充值上限控制） · 医疗记录子系统（record.c：诊断/处方 CRUD / 医生端三重权限校验 / 就诊状态硬约束流转 / 取消挂号的精确退费） · 患者自助服务（patientSelfService：登录 / 挂号 / 预约 / 查看记录 / PIN 验证链路） · 参与科室与床位管理模块（dept_bed.c）功能开发 · 编写患者模块、诊断模块、边界场景全部测试用例	25%	

学号	姓名	性别	班级	具体任务分工	所占比例	成绩
55250130	赵子睿	男	01	业务模块辅助开发 · 参与科室管理模块（ <code>dept_bed.c</code> ：科室 CRUD / 床位状态机 / 住院出院）功能开发 · 参与医生信息管理模块（ <code>doctor.c</code> ：医生 CRUD / 账号密码管理）功能开发 · 协助排班模块（ <code>schedule.c</code> ）功能测试与 bug 反馈 · 协助完成测试报告中的数据完整性验证章节	15%	

目录

- 一、项目概述
- 二、系统架构
- 三、数据模型
- 四、三角色运转流程
- 五、核心业务数据联动
- 六、安全与校验机制
- 七、问题解决策略
- 八、特色代码设计
- 九、相关测试数据
- 十、流程图演示
- 十一、相关界面展示
- 十四、项目不足与改进
- 后记

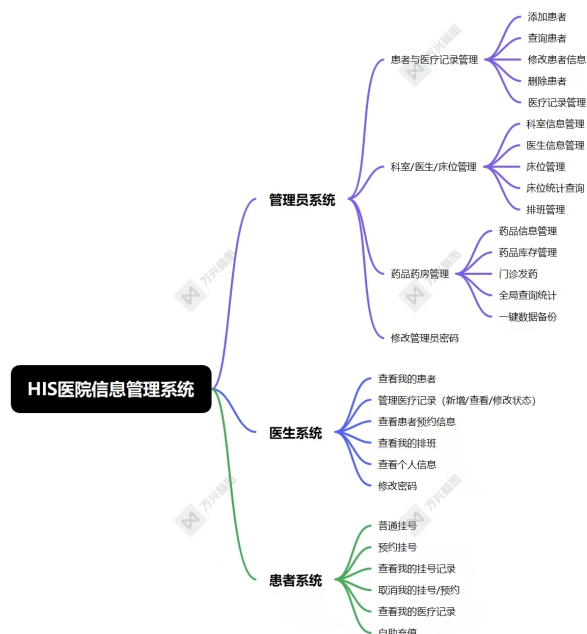
一、项目概述

1、系统概述

HIS医院信息系统核心功能基于 C99 标准（MSVC v143 编译器）实现，界面基于 Windows 控制台实现；数据在内存中以 通用单向链表（void 泛型）* 管理，通过 文本文件（|分隔字段）持久化存储，采用 write-then-rename 原子写入 机制保证数据一致性。

2.模块划分

系统分为 基础层 与 业务层 两个主要模块，基础层 提供通用的数据结构、工具函数和全局配置，业务层 按医院业务领域拆分为 10 个独立模块，每个模块负责特定的业务实体管理。



二、系统架构

2.1 源文件清单

类型	文件	行数	职责
配置	his_config.h	~90	长度常量、枚举、文件路径、默认值
配置	his.h	~230	8 个业务结构体、所有函数声明、extern 全局变量
基础	his_link.c	~110	通用链表： InitList/InsertNode/DeleteNode/FindNode/TraverseList/FreeList
基础	his_tool.c	~190	输入校验、ID 生成、文件 I/O、密码混淆、安全宏
主控	his_main.c	~500	程序入口 main()、三种角色登录认证、主菜单路由
业务	dept_bed.c	~530	科室 CRUD、床位 CRUD（含住院出院）、床位统计
业务	doctor.c	~390	医生 CRUD、账号密码管理、持久化
业务	schedule.c	~430	排班 CRUD、selectDoctorByDept、持久化
业务	drug.c	~550	药品 CRUD、库存管理（入库 / 出库 / 预警）、门诊发药
业务	admin_tools.c	~220	全局查询统计、管理员报表导出、一键数据备份
业务	patient.c	~630	患者 CRUD、自助充值、只读查看、PIN 校验
业务	record.c	~520	医疗记录 CRUD、医生工作站（查看患者 / 写诊断 / 开处方）
业务	registration.c	~500	普通挂号、预约挂号、查看 / 取消挂号

2.2 模块依赖关系

```
1 his_config.h (配置常量/枚举/路径)
2 |— his.h (结构体/函数声明 / extern 全局变量)
3 |
4 |— his.h (全局头文件)
5     |— his_main.c      (主控/菜单)
6     |   |— 业务层 (10 个文件)
7     |       |— dept_bed.c
8     |       |— doctor.c
9     |       |— schedule.c
10    |       |— drug.c
11    |       |— admin_tools.c
12    |       |— patient.c
13    |       |— record.c
14    |       |— registration.c
15    |
16    |— 业务模块 (8 文件)
17    |— his_tool.c      (工具函数)
18    |— his_link.c      (通用链表)
19    |   |— 8 个 LinkList 实例
```

2.3 跨模块函数调用

```
1 his_main.c
2 |— adminMenu()
3 |   |— patientModule() → patient.c
4 |       |— inputAndXxxRecords() → record.c
5 |   |— dept_bedModule() → dept_bed.c
6 |       |— doctorSubMenu() → doctor.c
7 |       |— scheduleSubMenu() → schedule.c
8 |   |— drugModule() → drug.c
9 |       |— globalStatsSubMenu() → admin_tools.c
10 |       |— backupAllData() → admin_tools.c
11 |— doctorMenu()
12 |   |— queryPatientByDoctor() → record.c
13 |   |— medicalRecordModule() → record.c
14 |   |— queryMyAppointment() → record.c
15 |— patientMenu()
16 |   |— normalRegistration() → registration.c
17 |   |— appointmentRegistration() → registration.c
18 |   |— patientRecharge() → patient.c
19 |   |— viewMyRegistration() → registration.c
20 |   |— cancelMyRegistration() → registration.c
```

所有跨模块调用通过 his.h 中的 extern 声明完成，模块间不直接引用对方内部函数。

三. 数据模型与关联关系

3.1 全局链表数据总线

链表	数据类型	写入模块	读取模块
patient_list	Patient	patient.c	registration.c, record.c, admin_tools.c
doctor_list	Doctor	doctor.c	schedule.c, registration.c, record.c, admin_tools.c
dept_list	Department	dept_bed.c	doctor.c, drug.c, schedule.c, registration.c, admin_tools.c
bed_list	Bed	dept_bed.c	admin_tools.c
drug_list	Drug	drug.c	admin_tools.c
record_list	MedicalRecord	record.c	patient.c, registration.c, admin_tools.c
schedule_list	DoctorSchedule	schedule.c	registration.c, admin_tools.c
appointment_list	Appointment	registration.c	schedule.c

3.2 实体关联关系

关系	类型	说明
Doctor.dept_id → Department.id	多对一	一个科室有多名医生
Bed.dept_id → Department.id	多对一	一个科室有多个床位
Patient.doctor_id → Doctor.id	多对一	一个医生有多个患者
MedicalRecord.patient_id → Patient.id	多对一	一个患者有多条医疗记录
Appointment.schedule_id → DoctorSchedule.id	多对一	一个排班有多个预约

3.3 相关数据结构设计

3.3.1 Patient

1	typedef struct {	
2	char id[MAX_ID_LEN];	// P260511001（前缀P+年月日+序号）
3	char name[MAX_NAME_LEN];	// 姓名
4	int age;	// 年龄
5	char gender[10];	// 男/女
6	float insurance_ratio;	// 医保报销比例（默认 0.7）
7	long long balance;	// 账户余额（分，避免浮点误差）
8	int is_inpatient;	// 枚举 PATIENT_OUT=0 / PATIENT_IN=1
9	char bed_id[MAX_ID_LEN];	// 绑定床位ID（住院时）
10	int record_count;	// 关联医疗记录数（冗余，用于快速统计）
11	char phone[15];	// 手机号
12	char id_card[20];	// 身份证号
13	char doctor_id[MAX_ID_LEN];	// 挂号医生ID（业务关键外键）
14	char dept_id[MAX_ID_LEN];	// 挂号科室ID

```

15     int register_status;           // 枚举 NONE=0 / PENDING=1 /
    IN_PROGRESS=2 / DONE=3
16     char register_time[25];       // 挂号时间
17     char pin[7];                  // 6 位访问密码（空串=未设置）
18     char register_record_id[MAX_ID_LEN]; // 关联的挂号记录ID（精确退款用）
19 } Patient;

```

3.3.2 Doctor

```

1  typedef struct {
2      char id[MAX_ID_LEN];          // D260511001
3      char name[MAX_NAME_LEN];      // 姓名
4      char dept_id[MAX_ID_LEN];     // 所属科室ID（→Department.id 外键）
5      char specialty[100];          // 擅长领域
6      char account[MAX_NAME_LEN];   // 登录账号（唯一性检查）
7      char password[MAX_PWD_LEN];   // 登录密码（nibble-swap 混淆存储）
8      int max_register;              // 每日最大挂号量
9      int current_register;          // 当前已挂号量
10     char register_date[11];        // 挂号日期 YYYY-MM-DD（用于每日重置）
11 } Doctor;

```

设计细节：register_date 与 current_register 配合实现“每日挂号量重置”——每次挂号时对比当前日期，若不同则将 current_register 置 0 并更新日期。

3.3.3 Department

```

1  typedef struct {
2      char id[MAX_ID_LEN];          // K260511001
3      char name[MAX_NAME_LEN];      // 科室名称
4      int doctor_count;              // 医生数
5  } Department;

```

3.3.4 Bed

```

1  typedef struct {
2      char id[MAX_ID_LEN];          // B260511001
3      RoomType room_type;           // 枚举 ROOM_NORMAL=1 / ROOM_SEMI=2 /
    ROOM_VIP=3
4      char dept_id[MAX_ID_LEN];     // 所属科室ID
5      BedStatus status;              // 枚举 BED_FREE=0 / BED_OCCUPIED=1
6      char patient_id[MAX_ID_LEN];  // 占用患者ID
7      char admit_time[30];          // 入院时间
8  } Bed;

```

状态机：空闲 → 入住（被占用） → 出院 → 空闲，严格单向流转。

3.3.5 Drug

```

1 typedef struct {
2     char id[MAX_ID_LEN];           // M260511001
3     char general_name[50];         // 通用名
4     char trade_name[50];           // 商品名
5     char alias[50];                 // 别名
6     float price;                     // 单价
7     int stock;                       // 库存
8     int warning_threshold;          // 预警阈值（低于此值触发预警）
9     char dept_id[MAX_ID_LEN];       // 所属科室ID
10 } Drug;

```

校验设计：general_name / trade_name / alias 三个名称字段各自用 while(1) 循环校验，禁止空输入和 | 分隔符。

3.3.6 MedicalRecord

```

1 typedef struct {
2     char id[MAX_ID_LEN];           // R260511001
3     char patient_id[MAX_ID_LEN];    // 关联患者ID
4     char doctor_id[MAX_ID_LEN];     // 负责医生ID
5     RecordType type;                // 枚举 1=挂号 2=诊断 3=检查 4=住院 5=
处方
6     long long cost;                  // 费用（分，避免浮点误差）
7     char detail[200];                // 详情
8     char create_time[30];            // 创建时间
9     int cancelled;                   // 是否已取消退款（0=正常，1=已取消）
10 } MedicalRecord;

```

作用：充当“审计日志”——每笔挂号、诊断、发药等操作都创建一条记录，患者端通过 PIN 验证后可查阅全部记录。

3.3.7 DoctorSchedule

```

1 typedef struct {
2     char id[MAX_ID_LEN];           // S260511001
3     char doctor_id[MAX_ID_LEN];    // 医生ID
4     char dept_id[MAX_ID_LEN];       // 科室ID
5     char date[11];                  // 日期 YYYY-MM-DD
6     char time_slot[20];             // 时间段（如"上午""下午"）
7     int max_patients;                // 总号源（受 Doctor.max_register 上限约
束）
8     int current_patients;            // 已预约数
9     int is_available;                // 是否可预约
10 } DoctorSchedule;

```

约束设计：addSchedule 时输入的 max_patients 必须 ≤ 所选 Doctor 的 max_register 字段。

3.3.8 Appointment


```
1 typedef struct {
2     char id[MAX_ID_LEN];           // A260511001
3     char patient_id[MAX_ID_LEN];   // 患者ID
4     char schedule_id[MAX_ID_LEN];  // 排班ID (→DoctorSchedule.id 外键)
5     char status[20];               // "已预约" / "已完成" / "已取消"
6     char create_time[30];          // 预约时间
7     long long cost;                 // 支付金额 (分, 避免浮点误差)
8 } Appointment;
```

3.4 枚举与配置层 (his_config.h)

枚举名	枚举值定义	说明
BedStatus	FREE=0, OCCUPIED=1	床位状态
InpatientStatus	OUT=0, IN=1	患者住院状态
RoomType	NORMAL=1, SEMI=2, VIP=3	病房类型
RecordType	REGISTER=1, DIAGNOSIS=2, EXAM=3, INHOSP=4, PRESCR=5	医疗记录类型
RegStatus	NONE=0, PENDING=1, IN_PROGRESS=2, DONE=3	挂号状态

ID 前缀体系 (8 个常量) :

前缀	实体	示例	备注
P	Patient	P260511001	患者 ID
D	Doctor	D260511001	医生 ID
K	Department	K260511001	科室 ID (取“科”拼音首字母)
B	Bed	B260511001	床位 ID
M	Drug	M260511001	药品 ID (取 Medicine 首字母)
R	MedicalRecord	R260511001	病历 ID
S	DoctorSchedule	S260511001	排班 ID (与 Patient 的 P 区分)
A	Appointment	A260511001	预约 ID

```
1 ID 生成规则: [前缀字符][年2位][月2位][日2位][序号3位], 例如 P260511001 = 患者 + 2026
   年05月11日 + 001号。序号每日自 001 重新开始 (GenerateID函数内部判断当日已生成数 + 1)。
```

3.5 文件序列化设计 (函数指针策略模式)

```

1 // 保存: 遍历链表 → format_func 将结构体格式化为 | 分隔行 → fprintf
2 int SaveDataToFile(LinkList* list, const char* filename,
3                   void (*format_func)(void*, char*));
4
5 // 加载: fgets 读取每行 → parse_func 解析 | 分隔行为结构体 → InsertNode
6 int LoadDataFromFile(LinkList* list, const char* filename,
7                      void (*parse_func)(char*, void*));

```

每个模块提供自己的 formatXxxLine() / parseXxxLine() 回调, 以 patient 为例:

```

1 // data/patient.txt 行格式 (| 分隔):
2 P260511001|张三|30|
  男|0.70|197.00|0||2|13800138000|110101199003071234|D260511001|K260511001|1|20
  26-05-11 09:30|123456

```

原子写入模式: 先写 .tmp → fclose → remove 原文件 → rename tmp 为原文件名, 避免写入中途崩溃导致文件损坏。

四. 三角色运转流程

4.1 系统初始化 (管理员)

4.1.1 程序启动流程

```

1 main()
2   └─ initGlobalLists()  — 创建 8 个空链表
3   └─ loadAllHisData()   — 从 data/*.txt 加载所有数据
4     └─ loadDeptData()   data/dept.txt
5     └─ loadDoctorData() data/doctor.txt
6     └─ loadBedData()    data/bed.txt
7     └─ loadPatientData() data/patient.txt
8     └─ loadDrugData()   data/drug.txt
9     └─ loadRecordData() data/record.txt
10    └─ loadScheduleData() data/schedule.txt
11    └─ loadAppointmentData() data/appointment.txt
12  └─ 主菜单循环
13      └─ 1. 管理员登录      ← 首次启动选择此项
14      └─ 2. 医生登录
15      └─ 3. 患者操作
16      └─ 0. 退出系统

```

首次启动时所有 data/*.txt 不存在, 8 个全局链表均为空, 系统处于"待初始化"状态。管理员登录后依次建科室、建医生、建床位、建药品。

4.1.2 管理员登录

账号 admin 硬编码在 his_config.h 中, 默认密码 123456。支持运行时修改密码, 持久化存储在 data/admin.dat。连续失败 5 次锁定, 重启程序后恢复。

4.1.3 添加科室

```
1 dept_bedModule() → deptSubMenu() → addDept()
2   1. inputDeptInfo: 输入科室名称 (while 循环校验非空 + 禁止 | 分隔符)
3   2. GenerateID: 生成 "K260511001" (K + 年月日 + 序号)
4   3. doctor_count = 0
5   4. InsertNode(dept_list)
6   5. saveDeptData() → 写入 data/dept.txt
```

4.1.4 添加医生

```
1 doctorSubMenu() → addDoctor()
2   1. 选择所属科室 (显示科室列表 → 输入科室ID → FindNode 校验)
3   2. inputDoctorInfo: 姓名 (while循环) / 特长 / 登录账号 (账号唯一性检查)
4   3. 输入密码 (非空校验 → passwordObfuscate nibble-swap 混淆)
5   4. 输入每日最大挂号量 (默认 30)
6   5. GenerateID → "D260511001"
7   6. InsertNode(doctor_list) → dept->doctor_count++
8   7. saveDoctorData() + saveDeptData()
```

4.1.5 添加床位与药品

床位: 选择所属科室 → 选择病房类型 (普通/双人/VIP) → GenerateID → status=BED_FREE → save

药品: 选择所属科室 → 输入通用名/商品名/别名/单价 → 初始库存/预警阈值 → GenerateID → save

库存预警: 预警阈值默认按库存的 20% (DRUG_WARNING_RATIO) 自动计算, 用户可修改。库存低于阈值时系统自动提示。

4.2 日常运转 (三角色协作)

4.2.1 场景一: 患者首次就诊 (普通挂号)

```
1 患者"张三"首次来医院, 挂内科张明医生的号
2
3 ① 患者菜单 → 1. 普通挂号
4   normalRegistration()
5   └─ inputAndFindOrCreatePatient()
6       └─ 输入手机号 13800138000 → 按ID查找 → 按手机号查找 → 未找到
7       └─ 询问是否创建新患者 → 是
8       └─ 姓名: 张三 | 年龄: 30 | 性别: 男
9       └─ 手机号: 13800138000 | 身份证: 110101199003071234
10      └─ (ValidateIDCard 18位加权校验 + isIDCardusedByOther 唯一性检查)
11      └─ 访问密码: 123456 (可选)
12      └─ GenerateID → "P260511001"
13      └─ savePatientData()
14
15 ② 余额不足, 先充值
16   患者菜单 → 6. 自助充值
17   └─ patientRecharge(): 充值 200 元 → 余额 200.00
18
19 ③ 再次挂号 (手机号匹配到已有患者)
20   selectDeptAndDoctor()
21   └─ 显示科室列表 → 选择 1.内科 → 显示内科医生
22   └─ 选择 1.张明 (号源 0/30, 充足)
23
```

```
24 ④ 确认费用 → 执行挂号
25   挂号费 10.00 | 医保报销 70% | 实际支付 3.00 元
26   余额 200.00 >= 3.00 → 通过
27   └─ 扣费: 200.00 → 197.00
28   └─ register_status = REG_STATUS_PENDING (待就诊)
29   └─ 张明.current_register: 0 → 1
30   └─ 创建挂号记录 R260511001 (RECORD_REGISTER, 3.00元)
31   └─ savePatientData() + saveDoctorData() + saveRecordData()
```

4.2.2 场景二：医生接诊

```
1  医生"张明"登录系统
2
3  ① doctorLogin()
4    └─ 账号: zhangming | 密码: 123456
5        └─ passwordObfuscate 混淆后比对通过
6        └─ 设置 g_current_doctor_id / g_current_doctor_name (会话跟踪)
7
8  ② 查看患者 (菜单 → 1.查看我的患者)
9    └─ 当前挂号: P260511001 张三 | 状态: 待就诊
10
11 ③ 写诊断记录 (菜单 → 2.管理医疗记录 → 2.新增诊断记录)
12   └─ 患者ID: P260511001
13   └─ 三重校验: 患者存在 → 已挂号(REG_STATUS_PENDING) → 挂的是自己的号
14   └─ 诊断: "患者血压偏高...诊断为轻度高血压" | 费用: 50.00
15   └─ 创建记录 R260511002 | saveRecordData()
16
17 ④ 修改就诊状态 (菜单 → 4.修改就诊状态)
18   └─ 待就诊 → 已完成 (状态流转硬约束)
19   └─ savePatientData()
```

4.2.3 场景三：门诊发药

```
1  药房管理员为张三发药 (阿莫西林胶囊 x2)
2
3  drugModule() → 3.门诊发药 → issuePrescription()
4
5  ① 校验患者 → 选药品 → 输入数量
6    患者ID: P260511001 → 找到张三 (余额 197.00)
7    药品ID: M260511001 → 阿莫西林 | 库存 500
8    数量: 2 | 库存 500 >= 2 → 通过
9    检查重复处方: hasDuplicatePrescription() → 无重复
10
11 ② 校验医生 → 计算费用
12   医生ID: D260511001 → 校验: 患者挂的张明的号
13   总费用: 12.50 x 2 = 25.00
14   医保报销: 25.00 x 70% = 17.50
15   自付: 25.00 - 17.50 = 7.50
16   余额 197.00 >= 7.50 → 通过
17
18 ③ 确认发药 → 执行
19   扣库存: 500 → 498 | 扣余额: 197.00 → 189.50
20   创建处方记录 R260511003 (RECORD_PRESCR, 25.00元)
21   saveRecordData() + saveDrugData() + savePatientData()
22   (保存顺序: 记录作为审计证据优先)
```

4.2.4 场景四：预约挂号

```
1  患者李四预约皮肤科林泽宇医生（2026-05-12 上午）
2
3  ① 前置：管理员新增排班
4      scheduleSubMenu() → 1.新增排班
5      └─ 选择科室：皮肤科 | 选择医生：林泽宇（selectDoctorByDept）
6      └─ 日期：2026-05-12 | 时段：上午
7      └─ 号源数：30（受医生 max_register 上限约束）
8      └─ GenerateID → "S260511001" | saveScheduleData()
9
10 ② 患者预约
11      appointmentRegistration()
12      └─ 创建患者：李四 | P260511002 | 充值 100 元
13      └─ 选择科室：皮肤科 | 选择医生：林泽宇
14      └─ 选择排班：2026-05-12 上午（号源 0/30）
15      └─ 预约费 20.00 | 医保 70% | 实付 6.00
16      └─ 余额 100.00 >= 6.00 → 通过
17      └─ 创建预约 A260511001 | status="已预约"
18      └─ saveAppointmentData() + saveScheduleData() + savePatientData()
19
20 ③ 取消预约（患者自助）
21      └─ PIN 验证 → 预约 status = "已取消"
22      └─ 排班号源恢复：current_patients: 1 → 0
23      └─ 退费：balance += cost
```

4.2.5 场景五：患者自助操作

```
1  查看医疗记录（菜单 → 5.查看我的医疗记录）
2      └─ 输入患者ID → PIN 码验证（3 次尝试）
3      └─ 显示个人信息 + 全部医疗记录明细
4
5      记录ID          | 类型 | 费用   | 详情
6      R260511001      | 挂号 | 3.00   | 普通挂号 - 费用：3.00
7      R260511002      | 诊断 | 50.00  | 诊断为轻度高血压
8      R260511003      | 处方 | 25.00  | 阿莫西林 x2，医保报销17.50元
9
10 自助充值（菜单 → 6.自助充值）
11      └─ 输入患者ID → PIN 验证
12      └─ 单次充值上限 100000 元 | 总额上限 500000 元
```

4.3 三种视角的功能矩阵

功能模块	管理员	医生	患者
系统初始化	添加科室 / 医生 / 床位 / 药品	-	-
患者管理	增删改查患者	查看挂自己号的患者	查看自己的信息
挂号	-	-	普通挂号 + 预约挂号
医疗记录	管理所有记录	写诊断 / 处方 / 改就诊状态	只读查看 (PIN 保护)
发药	门诊发药 (含医保结算)	-	-
药品库存	入库 / 出库 / 预警	-	-
床位管理	住院 / 出院办理	-	-
排班管理	增删改查排班	查看我的排班	-
充值	-	-	自助充值
取消挂号	-	-	取消挂号 / 预约 (退费)
统计报表	全院统计 (含文件导出)	-	-
数据备份	一键备份所有模块	-	-
账号管理	添加 / 修改医生账号	修改自身密码	-
个人信息	-	查看资料 + 修改密码	PIN 保护查看

五、核心业务数据联动

HIS 系统的核心操作均涉及多表（链表）联动更新。以下分析四个关键流程的数据变动。

5.1 挂号流程（涉及 patient + doctor + record）

```
1  normalRegistration()
2  |
3  |— 操作前状态
4  |   patient: 张三, balance=200.00, register_status=NONE
5  |   doctor:  张明, current_register=0/30
6  |
7  |— 步骤1: 选择科室+医生
8  |   内科 → 张明 (号源 0/30 充足)
9  |
10 |— 步骤2: 计算并校验
11 |   挂号费 10.00, 医保 70%, 自付 3.00
12 |   余额 200.00 >= 3.00 ✓
13 |
14 |— 步骤3: 扣费更新患者
15 |   balance: 20000→19700(分)  doctor_id 赋值
16 |   register_status→PENDING  record_count++
17 |
18 |— 步骤4: 更新医生号源
19 |   current_register: 0→1 (跨日自动清零)
```

```
20 |  
21 | └─ 步骤5：创建挂号记录  
22 |     MedicalRecord { type=REGISTER, cost=300(分),  
23 |                     detail="普通挂号 - 费用：3.00" }
```

数据联动表

文件	字段变化	操作
patient.txt	balance ↓, doctor_id 更新, register_status→1, record_count++	savePatientData()
doctor.txt	current_register++ (跨日检测自动重置)	saveDoctorData()
record.txt	新增 RECORD_REGISTER 记录 (含费用明细)	saveRecordData()

保存顺序：doctor → record → patient。医疗记录作为审计证据优先持久化。

5.2 发药流程 (涉及 drug + patient + record)

```
1  issuePrescription()  
2  |  
3  | └─ 步骤1：校验 (6 项)  
4  |     患者存在 ✓ | 药品存在 ✓ | 库存够 ✓  
5  |     医生存在 ✓ | 挂号匹配 ✓ | 重复处方警告 ✓  
6  |  
7  | └─ 步骤2：计算费用 (分精度)  
8  |     总价 = 12.50×2 = 25.00 元 = 2500(分)  
9  |     医保 = 2500×0.7 = 1750(分) | 自付 = 750(分)  
10 |  
11 | └─ 步骤3：执行更新  
12 |     drug.stock: 500→498  
13 |     patient.balance: 19700→18950(分)  
14 |     record_count: 2→3  
15 |  
16 | └─ 步骤4：创建处方记录  
17 |     MedicalRecord { type=PRESCR, cost=2500(分),  
18 |                     detail="门诊发药：阿莫西林胶囊 x2" }
```

数据联动表

文件	字段变化	操作
drug.txt	stock: 500→498	saveDrugData()
patient.txt	balance: 19700→18950(分), record_count++	savePatientData()
record.txt	新增 PRESCR 记录 (费用、药品明细)	saveRecordData()

保存顺序：record → drug → patient。先存处方记录 (审计证据)，再存库存和余额变更。

5.3 预约 / 取消预约（涉及 patient + schedule + appointment）

1	预约流程	取消预约流程
2		
3	patient.balance ↓	PIN 验证 ✓
4	schedule.current_patients++	a->status = "已取消"
5	新增 Appointment 记录	schedule.current_patients--
6	saveAppointmentData()	patient.balance += cost（退费）
7	saveScheduleData()	saveAppointmentData()
8	savePatientData()	saveScheduleData()
9		savePatientData()

5.4 住院 / 出院（涉及 bed + patient）

1	住院（BED_FREE→BED_OCCUPIED）	出院（BED_OCCUPIED→BED_FREE）
2		
3	bed: status=1, patient_id 绑定	bed: status=0, patient_id="-1"
4	patient: is_inpatient=1, bed_id 绑定	patient: is_inpatient=0, bed_id 清空
5	saveBedData() + savePatientData()	saveBedData() + savePatientData()

5.5 科室关联变更（涉及 dept + doctor）

操作	联动效果
添加医生	dept.doctor_count++
删除医生	dept.doctor_count--
删除科室(前置)	检查 doctor_count>0 和床位存在性

六、安全与校验机制

6.1 输入校验体系

系统构建了多层输入防护，从底层输入函数到业务校验逐层拦截非法数据：

层级一：统一菜单输入 — getValidChoice

1	int getValidChoice(int min, int max) {
2	char buf[64];
3	while (1) {
4	if (!inputLine(buf, sizeof(buf))) continue;
5	if (strlen(buf) == 0) continue;
6	int valid = 1;
7	for (int i = 0; buf[i]; i++)
8	if (buf[i] < '0' buf[i] > '9') { valid = 0; break; }
9	if (!valid) { printf("输入无效，只能输入数字! \n"); continue; }
10	int choice = atoi(buf);
11	if (choice >= min && choice <= max) return choice;
12	printf("输入无效，请输入 %d ~ %d 之间! \n", min, max);
13	}
14	}

对比裸 scanf("%d") 的常见问题：

问题	scanf	getValidChoice
输入"abc"	scanf 返回 0，缓冲区残留，后续 scanf 全部失败，陷入死循环	fgets 读整行，检测到非数字字符后提示重输，缓冲区已清空
输入"3 5"	scanf 只读 3，"\n" 和 "5" 残留，下次读取直接返回 5，造成逻辑错误	fgets 读整行，atoi 取第一个数字，无关字符不影响下次输入
缓冲区溢出	无保护，超长输入篡改栈上变量	fgets 限制 64 字符，超长部分留在 stdin 中由 ClearInputBuffer 清空

层级二：fgets 溢出检测与 GBK 截断保护

各模块的输入函数在 fgets 后检测末尾是否有换行符；若没有则说明输入超长，调用 ClearInputBuffer() 清空残留，并检测末尾字节是否 ≥0x81（GBK 中文字节范围），若是则移除该字节防止乱码：

```
1  if (!fgets(buf, sizeof(buf), stdin)) { ClearInputBuffer(); continue; }
2  char* nl = strchr(buf, '\n');
3  if (nl) {
4      *nl = '\0';          // 正常
5  } else {
6      ClearInputBuffer(); // 超长
7      size_t len = strlen(buf);
8      if (len > 0 && (unsigned char)buf[len - 1] >= 0x81)
9          buf[len - 1] = '\0'; // 移除孤立的 GBK 首字节
10 }
```

层级三：字段级校验

字段	校验规则	违规处理
姓名/科室名	不能包含分隔符 \	while 循环重输
年龄	0~150 整数	循环重输
性别	只能是"男"或"女"	循环重输
手机号	11 位、全数字、以 1 开头、全院唯一	格式或重复均重输
身份证	18 位、加权校验码 (GB 11643-1999)、全院唯一、年龄一致	逐个拦截重输
医保比例	0.0~1.0 浮点数	循环重输
余额/充值	单次 ≤ 10 万、总额 ≤ 50 万	提示上限后重输
菜单选项	数字、范围内	提示范围后重输

6.2 身份证校验 (GB 11643-1999)

```
1 int validateIDCard(const char* id_card) {
2     if (!id_card || strlen(id_card) != 18) return 0;
3     for (size_t i = 0; i < 17; i++)
4         if (id_card[i] < '0' || id_card[i] > '9') return 0;
5     char last = id_card[17];
6     if (!(last >= '0' && last <= '9') && last != 'x' && last != 'X') return
7     0;
8
9     static const int weights[17] = { 7,9,10,5,8,4,2,1,6,3,7,9,10,5,8,4,2 };
10    static const char check_chars[] = "10x98765432";
11    int sum = 0;
12    for (size_t i = 0; i < 17; i++)
13        sum += (id_card[i] - '0') * weights[i];
14
15    char expected = check_chars[sum % 11];
16    char actual_last = toupper(last); // x→X
17    return actual_last == expected;
18 }
```

前 17 位数字与加权系数相乘后累加，和模 11 得到校验位，与身份证末位比对。此算法来自国家标准 GB 11643-1999，可在不联网的情况下本地校验身份证真伪。

6.3 年龄与身份证联动校验

新增患者时，输入的年龄必须与身份证号中出生日期计算出来的年龄一致：

```
1 输入年龄：30
2 输入身份证：110101199003071234
3   → getAgeFromIDCard() 解析出生日期 1990-03-07
4   → 计算当前年龄 = 2026 - 1990 - (生日没过?) = 36
5   → 36 != 30 → 警告，重新输入
```

不一致时不会简单跳过，而是引导用户重新依次输入：

```
1 while (1) {
2     if (!inputAge(&p->age)) return 0;
3     if (inputIDCard(p->id_card, sizeof(p->id_card), NULL, p->age)) break;
4     printf("\n年龄和身份证号不一致，请重新输入.\n");
5 }
```

6.4 身份认证与访问控制

登录锁定

管理员和医生登录均有 `static` 变量记录连续失败次数，达到 5 次后锁定（程序重启后重置），防止暴力破解同时避免永久锁定维护问题。

```
1 adminLogin() / doctorLogin()
2     └─ static int fail_count
3         └─ fail_count >= 5 → [锁定] 登录尝试次数过多！
4         └─ 验证成功 → fail_count = 0, 返回 1
5         └─ 验证失败 → fail_count++, 返回 0
```

患者 PIN 码保护

敏感操作（查看记录、充值、取消预约）需要 6 位数字 PIN 码验证：

```
1 int verifyPatientPin(Patient* p) {
2     if (p->pin[0] == '\0') return 1; // 未设置则跳过
3     for (int tries = 0; tries < 3; tries++) {
4         if (strcmp(buf, p->pin) == 0) return 1;
5     }
6     printf("[错误] 验证失败已达上限，操作取消。\\n");
7     return 0;
8 }
```

- `pin[0] == '\0'` 表示未设置，向后兼容旧数据
- 3 次失败后取消操作，不暴露患者信息

医生端三重校验

```
1 // 写诊断/处方前的权限校验
2 if (!FindNode(patient_list, patient_id)) // 1. 患者存在
3 if (p->register_status == REG_STATUS_NONE) // 2. 已挂号
4 if (strcmp(p->doctor_id, doctor_id) != 0) // 3. 挂对号
```

角色权限隔离

功能	管理员	医生	患者
患者 CRUD	☒	☒	☒
医生 CRUD	☒	☒	☒
药品管理	☒	☒	☒
写诊断/处方	☒	☒	☒
查看患者	☒	☒ (仅挂自己号的)	☒ (仅自己，需PIN)
充值	☒	☒	☒ (需 PIN)
取消挂号	☒	☒	☒ (需 PIN)

6.5 密码存储安全 — nibble-swap

密码使用 nibble-swap（半字节交换）混淆存储：

```
1 void passwordObfuscate(char* pwd) {
2     for (int i = 0; pwd[i]; i++) {
3         pwd[i] = ((pwd[i] << 4) | ((unsigned char)pwd[i] >> 4));
4     }
5 }
```

自逆性：设一个字节的 nibble-swap 为 $f(x) = (L, H)$ ，则 $f(f(x)) = f(L, H) = (H, L) = x$ ，两次操作恢复原值。登录验证时只需对用户输入的密码做一次 obfuscate，再与文件中已混淆的密码直接 strcmp 比对。

安全等级：仅防“磁盘被直接读取时一目了然”，不防逆向工程。

6.6 文件写入安全 — 原子保存

```
1 int SaveDataToFile(LinkList* list, const char* filename,
2                     void (*format_func)(void*, char*)) {
3     char tmp[MAX_PATH];
4     snprintf(tmp, sizeof(tmp), "%s.tmp", filename);
5     // ① 写临时文件
6     FILE* fp = fopen(tmp, "w");
7     // ② 遍历链表写入
8     // ③ fclose
9     // ④ remove(原文件)
10    // ⑤ rename(tmp, 原文件) ← 原子操作
11 }
```

```
1 保存步骤:
2 ① fopen("data/patient.txt.tmp", "w")    - 写临时文件
3 ② 逐行写入临时文件
4 ③ fclose(tmp)
5 ④ remove("data/patient.txt")              - 删除原文件
6 ⑤ rename(tmp, "data/patient.txt")         - 原子重命名
```

`rename()` 是操作系统原语级别的原子操作（NTFS 保证）。如果程序在 ①~③ 之间崩溃，临时文件不完整但原文件完好；如果程序在 ④ 之后崩溃，临时文件已替代原文件。不存在“写到一半崩溃导致文件损坏”的窗口。

6.7 业务安全校验汇总

校验场景	校验内容	违规处理
删除科室	科室下医生数 > 0? 有床位?	提示无法删除
删除医生	有排班? 有患者挂号?	校验后拦截
删除患者	有医疗记录? 有预约? 在住院? 有挂号?	逐项检查后拦截
发药	患者存在? 药品存在? 库存够? 医生匹配? 余额足? 重复处方?	逐项拦截或警告
就诊状态流转	NONE→PENDING→IN_PROGRESS→DONE 硬约束	非法跳转拦截
号源限制	current_register < max_register	提示号源已满
跨日重置	挂号日期 != 今天?	自动清零号源

七、问题解决策略

7.1 密码明文迁移问题

问题：早期版本中 `loadDoctorData()` 通过 `isprint()` 判断加载的密码是否为明文。但 nibble-swap 混淆后的部分字节恰好落在可打印 ASCII 范围内（如 '6'→'6'、'3'→'3'），导致程序重启后误判，反复混淆已混淆的密码，数据彻底损坏。

```

1  错误判断（修复前）：
2      strcpy(check, d->password);
3      passwordObfuscate(check);          // 对已混淆密码再次混淆 = 还原
4      if (isprint(check[0])) {          // 混淆后某些字符仍可打印
5          strcpy(d->password, check);    // 明文→混淆 ✓ | 混淆→还原 ✗
6      }

```

修复：将密码迁移逻辑移至登录时按需处理，不再依赖 `isprint()`：

```

1  // doctorLogin 中的迁移逻辑
2  char check[MAX_PWD_LEN];
3  HIS_STRNCPY(check, d->password, MAX_PWD_LEN);
4  passwordObfuscate(check);
5  if (strcmp(check, password) == 0) {
6      // 匹配说明 d->password 是明文 → 覆盖为混淆版本
7      HIS_STRNCPY(d->password, password, MAX_PWD_LEN);
8      saveDoctorData();
9      return 1;
10 }
11 if (strcmp(d->password, password) == 0) {
12     return 1; // 已经是混淆版本，直接匹配
13 }

```

关键点：先对文件中读取的密码做 obfuscate，如果和用户输入（已 obfuscate）匹配，说明原密码是明文 → 覆盖为混淆版本；如果文件密码直接和用户输入匹配，说明已经是混淆版本 → 正常登录。

7.2 GBK 中文字符截断乱码

问题：fgets 在输入超长时可能截断在 GBK 中文首字节（0x81~0xFE），末尾留下孤立字节产生乱码。

```

1  输入："一二三四五六七八九十"（超长）
2  fgets 截断："一二四五\xB0" ← \xB0 是 GBK 首字节，单独显示为乱码

```

修复：截断后检查末尾字节 $\geq 0x81$ 则移除：

```

1  size_t len = strlen(buf);
2  if (len > 0 && (unsigned char)buf[len - 1] >= 0x81)
3      buf[len - 1] = '\0';

```

为什么 $\geq 0x81$ ？GBK 中文首字节为 0x81~0xFE，ASCII 可见字符最大 0x7E（'~'），所以 $\geq 0x81$ 的字符一定不是完整 ASCII 字符。

7.3 parseBedLine 类型转换安全修复

问题：直接对枚举指针做 `(int*)` 强制转换，属于 C 语言未指定行为：

```

1  // 修复前 - 未指定行为
2  sscanf(line, ..., (int*)&b->room_type, ..., (int*)&b->status, ...);

```

修复后 — 使用中间 int 变量：

```

1 | int room_type, status;
2 | sscanf(line, ..., &room_type, ..., &status, ...);
3 | b->room_type = (RoomType)room_type;
4 | b->status = (BedStatus)status;

```

同时修复了 sscanf 宽度控制防缓冲区溢出。

7.4 parseDeptLine 缓冲区溢出修复

问题：sscanf 的 %[^\n] 没有宽度限制：

```

1 | // 修复前 - 无宽度限制
2 | sscanf(line, "%[^\n]|%[^\n]|%d", d->id, d->name, &d->doctor_count);

```

修复后 — 精确宽度控制：

```

1 | sscanf(line, "%19[^\n]|%49[^\n]|%d", d->id, d->name, &d->doctor_count);

```

%19[^\n] 限制最多 19 字符 + 1 个 '\0' = 20 字符，适配 d->id[MAX_ID_LEN]。

7.5 发药数据保存顺序调整

问题：原流程先存库存变更，最后存处方记录：

```

1 | 原顺序: saveDrugData() → savePatientData() → saveRecordData()
2 | 如果第 1 步后崩溃 → 库存已扣但处方记录丢失，无法追溯

```

修复后：

```

1 | 新顺序: saveRecordData() → saveDrugData() → savePatientData()
2 |           ↑
3 |           审计证据优先持久化

```

即使后续保存失败，至少有一条处方记录可以追溯该操作。

7.6 就诊状态流转硬约束

问题：最初不检查当前状态，医生可直接从"待就诊"跳到"已完成"。

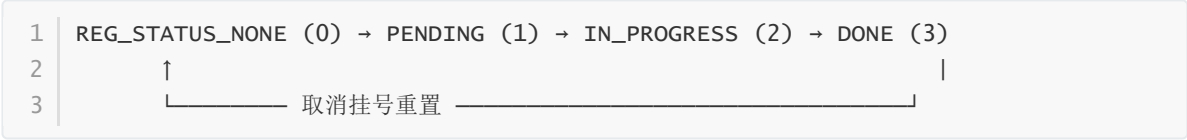
修复：增加状态跳转校验：

```

1 | if (st == 1) { // → 就诊中
2 |     if (p->register_status != REG_STATUS_PENDING) { /* 拦截 */ }
3 |     p->register_status = REG_STATUS_IN_PROGRESS;
4 | } else if (st == 2) { // → 已完成
5 |     if (p->register_status != REG_STATUS_IN_PROGRESS) { /* 拦截 */ }
6 |     p->register_status = REG_STATUS_DONE;
7 | }

```

状态流转图：



7.7 分精度计算（浮点误差问题）

问题：直接使用 `float` 计算金额会导致累积误差（如 $0.1 + 0.2 = 0.30000000000000004$ ），多次扣费后余额逐渐偏离。

修复：所有金额以"分"为单位存储（`long long`），仅显示时除以 100：

```
1 // 计算（整数精度）
2 long long total_cents = (long long)(price * quantity * 100.0 + 0.5);
3 long long insurance_cents = (long long)(total_cents * insurance_ratio);
4 long long patient_cents = total_cents - insurance_cents;
5
6 // 显示（临时转元）
7 printf("总费用: %.2f 元\n", (double)total_cents / 100.0);
```

注意 `+0.5` 用于四舍五入，避免 `(int)` 截断导致 1 分的精度损失。

八、特色代码设计

8.1 void* 泛型链表

所有数据在内存中统一以通用单向链表管理：

```
1 typedef struct ListNode {
2     void* data;           // 任意类型数据的指针
3     int data_size;        // 数据大小
4     char id[MAX_ID_LEN];  // 节点 ID（用于 O(n) 快速查找）
5     struct ListNode* next;
6 } ListNode;
7
8 typedef struct {
9     ListNode* head;
10    int length;
11 } LinkList;
```

8 个业务实体共用同一套链表操作：

函数	作用
<code>InitList()</code>	创建空链表
<code>InsertNode()</code>	插入节点（深拷贝）
<code>DeleteNode()</code>	按 ID 删除节点
<code>FindNode()</code>	按 ID 查找（O(n)）
<code>TraverseList()</code>	遍历打印
<code>FreeList()</code>	释放全部节点

对比方案：`void*` + 函数指针（本项目，类似 C 标准库 `qsort` 思路）vs. `#define` 宏模板（代码膨胀）vs. 每个实体手写一套链表（冗余）。

8.2 函数指针策略模式

数据持久化采用策略模式，将"格式化/解析"策略与"文件读写"骨架分离：

```
1 // 骨架 (his_tool.c, 固定不变)
2 int SaveDataToFile(LinkList* list, const char* filename,
3                   void (*format_func)(void*, char*));
4 int LoadDataFromFile(LinkList* list, const char* filename,
5                      void (*parse_func)(char*, void*));
6
7 // 策略示例 - patient.c 提供
8 static void formatPatient(void* data, char* line) {
9     Patient* p = (Patient*)data;
10    snprintf(line, MAX_LINE_LEN, "%s|%s|%d|%s|...", /* 各字段 */);
11 }
12 static void parsePatient(char* line, void* data) {
13     Patient* p = (Patient*)data;
14     // 按 | 拆分字段并赋值
15 }
```

8 个业务模块各提供一对 `format+parse` 回调，避免重复编写文件读写代码。

8.3 原子写入机制

```
1 保存步骤：
2 ① fopen("data/patient.txt.tmp", "w")    - 写临时文件
3 ② 遍历链表，逐行写入临时文件
4 ③ fclose(tmp)
5 ④ remove("data/patient.txt")              - 删除原文件
6 ⑤ rename(tmp, "data/patient.txt")         - 原子重命名（NTFS 保证）
```

`rename()` 是操作系统原语级别的原子操作。不存在"写到一半崩溃导致文件损坏"的窗口。

8.4 分精度金额计算

结构体中的金额字段全部以 `long long` 类型按"分"存储：

```
1 typedef struct {
2     long long balance;    // 余额（分）
3     // ...
4 } Patient;
5
6 typedef struct {
7     long long cost;       // 医疗费用（分）
8     // ...
9 } MedicalRecord;
```

为什么不用 `float`？`float` 精度约 7 位十进制，多次运算后误差累积。`long long` 以"分"为单位可精确表示 0~9e18 分范围，完全满足医院系统的金额精度要求。

8.5 ID 生成策略

```
1 void GenerateID(char* id, char type) {
2     static int seq = 1;
3     time_t t = time(NULL);
4     struct tm* tm = localtime(&t);
5     snprintf(id, MAX_ID_LEN, "%c%02d%02d%02d%03d",
6             type, tm->tm_year % 100, tm->tm_mon + 1, tm->tm_mday, seq++);
7 }
```

前缀	实体	示例	说明
P	Patient	P260511001	患者
D	Doctor	D260511001	医生
K	Department	K260511001	科室 ("科"拼音首字母)
B	Bed	B260511001	床位
M	Drug	M260511001	药品 (Medicine)
R	MedicalRecord	R260511001	病历
S	DoctorSchedule	S260511001	排班
A	Appointment	A260511001	预约

ID 格式：[前缀][年2位][月2位][日2位][序号3位]，序号每日自 001 重新开始。

8.6 HIS_STRNCPY 安全宏

```
1 #define HIS_STRNCPY(dst, src, cap) do { \
2     char* _d = (dst); \
3     size_t _c = (size_t)(cap); \
4     const char* _s = (src); \
5     if (_d && _c > 0U) { \
6         strncpy(_d, (_s) ? (_s) : "", _c - 1U); \
7         _d[_c - 1U] = '\0'; \
8     } \
9 } while (0)
```

`strncpy` 在源字符串长度 $\geq$ 目标大小时不自动添加 `'\0'`，这是 C 标准库的经典陷阱。此宏封装保证了：

- 1. 目标缓冲区始终以 `'\0'` 结尾
- 2. 源指针为 `NULL` 时拷贝空字符串而非崩溃
- 3. 目标容量为 0 时安全跳过

8.7 跨模块函数调用体系

所有跨模块调用通过 `his.h` 中的 `extern` 声明完成：

```
1 his_main.c
2 |─ patientModule()      → patient.c
3 |  |─ inputAndXxxRecords() → record.c
4 |─ dept_bedModule()    → dept_bed.c
5 |  |─ doctorSubMenu()    → doctor.c
6 |  |─ scheduleSubMenu()  → schedule.c
7 |─ drugModule()        → drug.c
8 |  |─ globalStatsSubMenu() → admin_tools.c
9 |  |─ backupAllData()    → admin_tools.c
```

各业务模块内部函数全部声明为 `static`，只暴露必要的入口函数。模块间不直接引用对方内部函数，实现了 C 语言级别的信息隐藏。

8.8 next_token 字段分割

```
1 // 按 '|' 分割，连续分隔符不会跳过（与 strtok 不同）
2 char* next_token(char** str) {
3     if (!str || !*str) return NULL;
4     char* start = *str;
5     char* p = strchr(start, '|');
6     if (p) { *p = '\0'; *str = p + 1; }
7     else { *str = NULL; }
8     return start;
9 }
```

配合 `sscanf` 用于 `schedule.c`/`drug.c` 的解析，或单独使用于 `patient.c`/`record.c` 等模块的解析。连续分隔符正确处理空字段，是向后兼容的关键。

九、相关测试数据

9.1 功能测试

系统经过完整的黑盒功能测试，覆盖 12 个业务模块，共 98 个测试用例，通过率 100%。

患者模块（TC-PAT-001 ~ 008）

用例	场景	结果
TC-PAT-001	正常录入患者（完整信息）	☑
TC-PAT-002	姓名为空拦截	☑
TC-PAT-003	姓名修改为 49 字符（MAX-1 临界值）	☑
TC-PAT-004	姓名输入 60 字符（超长截断）	☑
TC-PAT-005	身份证含特殊字符拦截	☑
TC-PAT-006	手机号不足 11 位拦截	☑
TC-PAT-007	批量患者列表展示	☑
TC-PAT-008	10 名同名患者"张三"共存	☑

挂号 & 诊断 & 其他模块

模块	用例范围	用例数	结果
挂号	TC-REG-001~009	9	☒ 全部通过
诊断/处方	TC-DIA-001~006	6	☒ 全部通过
住院	TC-INP-001~005	5	☒ 全部通过
床位	TC-BED-001~011	11	☒ 全部通过
排班	TC-SCH-001~009	9	☒ 全部通过
药品	TC-DRG-001~014	14	☒ 全部通过
权限控制	ACL-A-001~D-006	10	☒ 全部通过
全局统计	TC-STA-001~006	6	☒ 全部通过
数据备份	TC-BAK-001~003	3	☒ 全部通过
退出/持久化	TC-EXT-001~004	4	☒ 全部通过

9.2 边界场景测试

针对输入边界进行了专项测试，44 项全部通过：

类别	测试项数	通过率
姓名字段边界	5	100%
年龄字段边界	5	100%
手机号/身份证边界	5	100%
医保/余额边界	6	100%
号源与状态边界	7	100%
床位状态边界	8	100%
药品库存边界	8	100%
合计	44	100%

关键边界验证结果举例：

测试项	输入	预期	实际
年龄为 0	0	允许（新生儿）	☑
负年龄	-5	拦截	☑
超大年龄	200	拦截	☑
医保比例 >1.0	2.0	钳位为 1.0	☑
余额刚好等于挂号费	balance == pay	扣费后余额 0，挂号成功	☑
号源归零	current_register=0 且取消	检查 >0 后再减，不出负数	☑
库存为 0 时出库	stock=0	拦截"库存不足"	☑
取消已取消的预约	status="已取消" 再取消	提示"已取消"	☑

9.3 数据容量测试

批量操作测试结果：

数据类型	目标数量	实际创建	耗时
科室	5	5	<1s
医生	20	20	<2s
患者（含 10 名同名）	110	110	<5s
药品	20	20	<2s
床位（三种病房类型）	30	30	<2s

所有 ID 唯一性验证通过，同名患者各有唯一 ID。

容量推算：

场景	数据量	预估内存	预计响应
小型医院	1000 患者 / 5000 记录	<2 MB	即时
中型医院	10000 患者 / 50000 记录	<15 MB	<1s
大型医院	100000 患者 / 500000 记录	<150 MB	1~3s

9.4 数据完整性验证

通过操作前后对比数据文件内容，验证 **12 项**跨模块一致性：

场景	检查项	结果
挂号后	patient.txt balance 减少 ✓, doctor.txt current_register 增加 ✓	☑
发药后	drug.txt stock 减少 ✓, record.txt 新增 PRESCR 记录 ✓	☑
住院后	bed.txt status=1 ✓, patient.txt is_inpatient=1 ✓	☑
出院后	bed.txt status=0, patient_id=-1 ✓	☑
取消挂号后	doctor.txt current_register 递减 ✓	☑
取消预约后	schedule.txt current_patients 递减 ✓, 退费 ✓	☑

9.5 数据保存机制

存储概览

所有业务数据统一存储于 `data/` 目录下，每个链表对应一个文本文件，字段以 `|` 分隔：

文件	对应链表	行数	存储内容
<code>data/patient.txt</code>	patient_list	~110	患者基本信息
<code>data/doctor.txt</code>	doctor_list	~20	医生账号及号源
<code>data/dept.txt</code>	dept_list	~5	科室信息
<code>data/bed.txt</code>	bed_list	~30	床位状态及患者绑定
<code>data/drug.txt</code>	drug_list	~20	药品信息及库存
<code>data/record.txt</code>	record_list	~130	医疗记录（审计日志）
<code>data/schedule.txt</code>	schedule_list	~10	医生排班
<code>data/appointment.txt</code>	appointment_list	~10	预约记录

行格式示例

以 `data/patient.txt` 为例，每行对应一条完整记录：

1	P260511001 张三 30 男 0.70 19700 0 2 13800138000 110101199003071234 D260511001 K260511001 1 2026-05-11 09:30 123456 R260511001
---	--

数据生命周期

```
1  程序启动
2    └─ main()
3        │─ initGlobalLists()    ← 创建 8 个空链表
4        │─ loadAllHisData()     ← 从 data/*.txt 加载全部数据
5        │   └─ 文件不存在时静默跳过（首次启动场景）
6        │
7        │─ 业务操作
8        │   └─ 每步关键操作后调用 saveXxxData() ← 实时持久化
9        │
10       └─ 程序退出
11           └─ 不额外保存（每次 save 已实时写入）
```

- **首次启动：**data/ 目录为空，各 load 函数自动跳过，系统处于待初始化状态
- **运行过程：**每次挂号、发药、录入等操作后立即调用对应的 saveXxxData()，确保数据实时落地
- **原子写入：**saveXxxData() 内部采用 write-then-rename 模式——先写 .tmp 临时文件，成功后 remove() 原文件再 rename()，保证写入中途崩溃不会损坏原文件

运行前

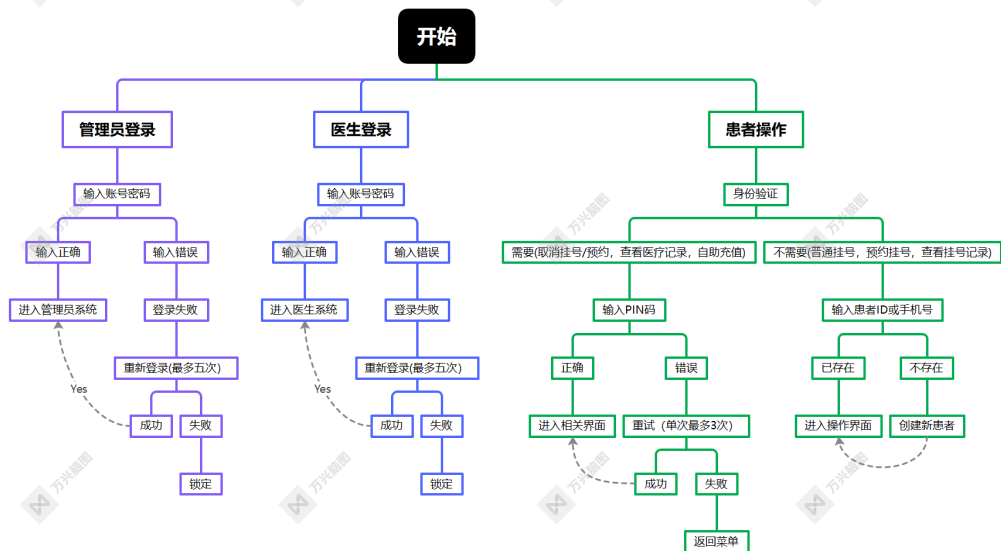
名称	修改日期	类型	大小
此文件夹为空。			

运行后

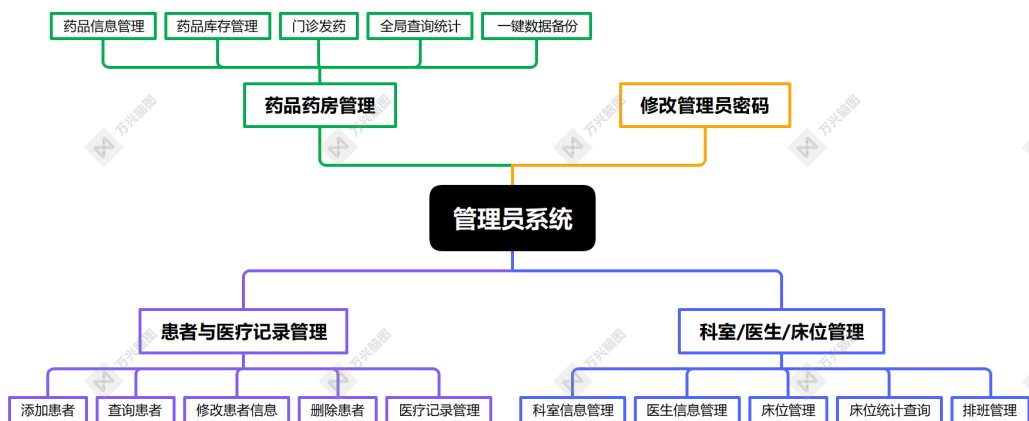
名称	修改日期	类型	大小
admin.dat	2026/5/14 9:54	DAT 文件	1 KB
appointment.txt	2026/5/17 11:36	文本文档	1 KB
bed.txt	2026/5/17 11:36	文本文档	2 KB
dept.txt	2026/5/17 11:36	文本文档	1 KB
doctor.txt	2026/5/17 11:57	文本文档	2 KB
drug.txt	2026/5/17 11:57	文本文档	1 KB
patient.txt	2026/5/17 11:57	文本文档	1 KB
record.txt	2026/5/17 11:57	文本文档	1 KB
schedule.txt	2026/5/17 11:36	文本文档	1 KB

十、流程图演示

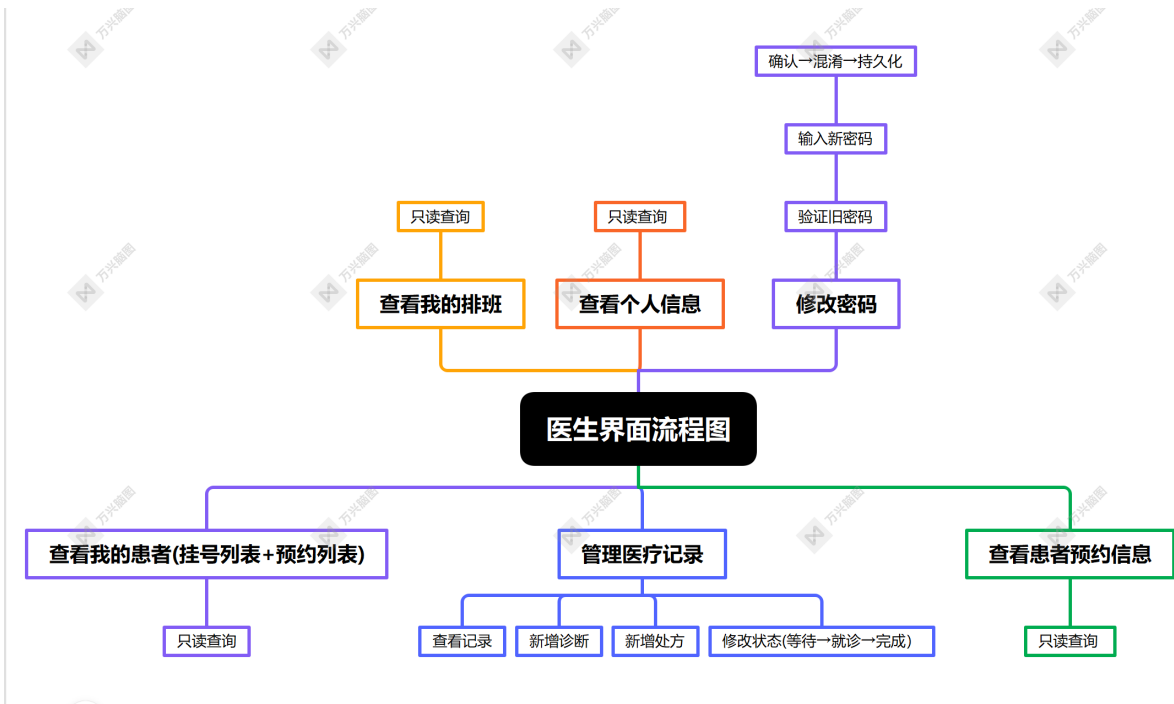
1.登录界面流程



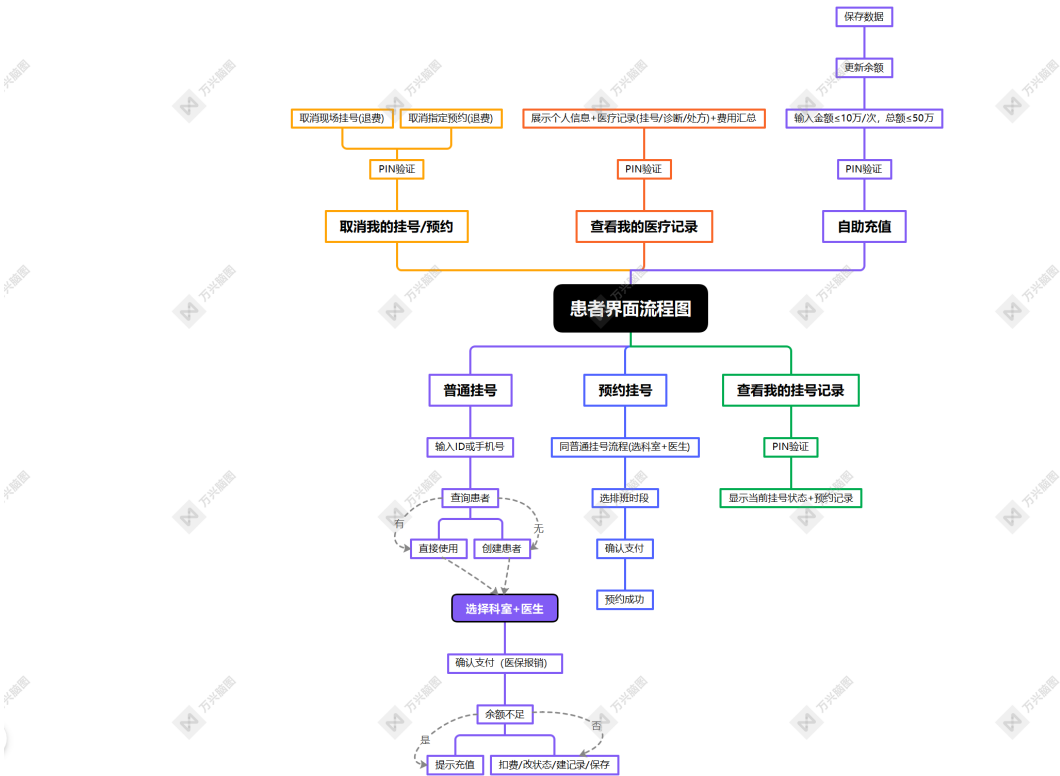
2. 管理员界面流程图



3. 医生界面流程图



4.患者界面流程图



十一、相关界面展示

1.初始界面


```
=====
                        HIS医院信息系统  [主菜单]
=====

1. 管理员登录
2. 医生登录
3. 患者操作
0. 退出系统

=====
请输入选择：
```

2.管理员登录界面

```
=====
                        管理员登录
=====

请输入账号： admin
请输入密码： 123456

=====
[登录成功] 欢迎使用HIS医院信息系统！
=====
```

3.管理员菜单

```
=====
                        管理员菜单
=====

1. 患者与医疗记录管理
2. 科室/医生/床位管理
3. 药品药房管理
4. 修改管理员密码
0. 返回主菜单

=====
请输入选择：
```

4.患者与医疗记录管理

===== 患者与医疗记录管理 =====

1. 添加患者
2. 查询患者
3. 修改患者信息
4. 删除患者
5. 医疗记录管理
0. 返回

请选择：

5.添加患者

```
请输入姓名：张三
请输入年龄（0-150）：15
请输入性别（男/女）：男
请输入医保报销比例（0.0~1.0）：0.5
请输入初始余额：100
请输入手机号（11位手机号）：15162482356
请输入身份证号（18位）：430102199503152476
身份证号格式错误，请输入18位有效身份证号！
请输入身份证号（18位）：110105200107081234
身份证出生日期计算年龄为24岁，与输入的15岁不符，请重新输入年龄和身份证号！
请输入年龄（0-150）：24
请输入性别（男/女）：男
请输入医保报销比例（0.0~1.0）：0.5
请输入初始余额：100
请输入手机号（11位手机号）：15162482356
请输入身份证号（18位）：110105200107081234
请设置6位访问密码（用于保护个人隐私，直接回车则不设置）：123456
```

[成功] 患者 张三（ID: P260516001）添加成功！

6.查询患者

6.1 查询菜单

```
===== 患者查询 =====
1. 按ID查询
2. 按姓名查询（模糊查询，支持重名）
3. 遍历所有患者
4. 按挂号科室查询
5. 按就诊状态查询
0. 返回
请选择：
```

6.2 按ID查询

```
请选择：1

请输入患者ID：P260516001
ID: P260516001 | 姓名：张三 | 年龄：24 | 性别：男 | 手机：15162482356 | 身份证：
110105200107081234 | 余额：100.00 | 医保：50% | 状态：未住院
```

6.3 按姓名查询（模糊查询，支持重名）

```
请选择：2

请输入患者姓名：张三
ID: P260516001 | 姓名：张三 | 年龄：24 | 性别：男 | 手机：15162482356 | 状态：
未挂号
共找到 1 位患者。
```

6.4 遍历所有患者

```
----- 患者列表 -----
ID: P260514001 | 姓名: 冯辉 | 年龄: 19 | 性别: 男 | 手机: 43110220061220 | 身份证: 0 | 余额: 100.00 | 医保: 50% | 状态: 未住院
ID: P260516001 | 姓名: 张三 | 年龄: 24 | 性别: 男 | 手机: 15162482356 | 身份证: 110105200107081234 | 余额: 100.00 | 医保: 50% | 状态: 未住院
```

6.5 按挂号科室查询

```
请选择: 4

请输入科室ID: K260514001

科室 K260514001 (皮肤科) 下挂号患者:
该科室下无挂号患者。
```

6.6 按就诊状态查询

```
请选择就诊状态:
1. 待就诊
2. 就诊中
3. 已完成
0. 取消
请选择: 1

--- 待就诊 患者 ---
无符合条件的患者。
```

7.修改患者信息

```
请选择: 3

请输入要修改的患者ID: P260514001

当前患者: 冯辉 (ID: P260514001)

--- 修改患者信息 ---
1. 姓名: 冯辉
2. 年龄: 19
3. 性别: 男
4. 医保比例: 0.50
5. 余额: 100.00
6. 手机号: 43110220061220
7. 身份证: 0
0. 保存并返回
请选择要修改的字段: 1
请输入新值: lisi

--- 修改患者信息 ---
1. 姓名: lisi
2. 年龄: 19
3. 性别: 男
4. 医保比例: 0.50
5. 余额: 100.00
6. 手机号: 43110220061220
7. 身份证: 0
0. 保存并返回
请选择要修改的字段:
```

8.删除患者

```
请选择: 4

请输入要删除的患者ID (输入0取消): P260514001
确认删除患者 lisi (ID: P260514001) ? (y/n): y

[成功] 患者 lisi 已删除!
```

9.医疗记录管理

9.1 菜单

```
--- 医疗记录管理 ---
1. 查看患者的医疗记录
2. 新增医疗记录
3. 修改医疗记录
4. 删除医疗记录
0. 返回上级
请选择：
```

9.2 查看患者的医疗记录

```
请选择：1
请输入患者ID：P260516001

--- 患者的医疗记录 ---
记录ID：R260516002 | 医生：张医生 | 类型：挂号 | 费用：10.00 | 时间：2026-05-16 16:24:09
详情：痤疮基础检查
```

9.3 新增医疗记录

```
请选择：2
请输入患者ID：P260516001
请输入医生ID：D260514008
请输入记录类型（1-挂号 2-诊断 3-检查 4-住院 5-处方）：1
请输入费用：10
请输入详情：痤疮基础检查
【警告】医生所属科室（K260514001）与患者科室（C）不匹配！
【成功】医疗记录 R260516002 添加成功！
```

9.4 修改医疗记录

```
请选择：3
请输入要修改的记录ID：R260516002
当前记录：R260516002（患者：P260516001，类型：1）
请输入新的详情（当前：痤疮基础检查）：痤疮基础复查，面部皮肤基础检查
请输入新的费用（当前：10.00）：20
请输入新的记录类型（1-挂号 2-诊断 3-检查 4-住院 5-处方）（当前：1）：2
【成功】记录已更新！
```

9.5 删除医疗记录

```
请选择：4
请输入要删除的记录ID（输入0取消）：R260516002
确认删除记录 R260516002？(y/n)：y
【成功】记录已删除！
```

10.科室/医生/床位管理模块

```
=====
                科室/医生/床位管理模块
=====

1. 科室信息管理
2. 医生信息管理
3. 床位管理
4. 床位统计查询
5. 排班管理
0. 返回主菜单

=====
请输入选项：
```

11. 科室信息管理

11.1 菜单

```
=====
          科室信息管理 【子菜单】
=====
1. 添加科室
2. 修改科室信息
3. 删除科室
4. 查询科室
0. 返回上级
=====
请输入选项：1
```

11.2 添加科室

```
请输入选项：1
请输入科室名称：儿科

[成功] 科室添加成功，科室ID：K260516003
```

11.3 修改科室信息

```
请输入选项：2

请输入需要修改的科室ID：K260516003

当前科室信息：
=====
科室ID：K260516003
科室名称：儿科
科室医生数：0
=====

--- 请输入新的科室名称 ---
请输入科室名称：妇科

[成功] 科室信息修改成功！
```

11.4 删除科室

```
请输入选项：3

请输入需要删除的科室ID（输入0取消）：K260516003

[确认] 确定要删除科室 妇科（K260516003）吗？(y/n): y

[成功] 科室删除成功！
```

11.5 查询科室

请输入选项：4

--- 科室查询 ---

1. 按ID精确查询
2. 列出所有科室

请选择：1

请输入科室ID：K260514001

--- 查询结果 ---

```
=====
科室ID: K260514001
科室名称: 皮肤科
科室医生数: 3
=====
```

12.医生信息管理

12.1 添加医生

```
=====
科室ID: K260514007
科室名称: 心血管内科
科室医生数: 0
=====

请输入所属科室ID: K260514007
请输入医生姓名: 文医生
请输入医生特长: 擅长高血压、高血脂、冠心病、心绞痛规范化诊疗与长期慢病管理
请输入登录账号: wen123
请输入登录密码: 123456
请输入每日最大挂号量 (默认30): 30

[成功] 医生添加成功, 医生ID: D260516004
```

12.2 修改医生信息

```
请输入选项：2

请输入需要修改的医生ID: D260516004

===== 修改医生信息 =====

[医生ID: D260516004]
姓名: 文医生      所属科室: 心血管内科
特长: 擅长高血压、高血脂、冠心病、心绞痛规范化诊疗与长期慢病管理

=====
1. 修改姓名 (当前: 文医生)
2. 修改所属科室 (当前: K260514007)
3. 修改擅长领域 (当前: 擅长高血压、高血脂、冠心病、心绞痛规范化诊疗与长期慢病管理)
4. 修改登录账号 (当前: wen123)
5. 修改登录密码
6. 修改每日最大挂号量 (当前: 30)
0. 保存并返回

=====
请选择要修改的字段：0

[成功] 医生信息修改成功!
```

12.3 删除医生

```
请输入选项：3

请输入需要删除的医生ID（输入0取消）：D260516004

[确认] 确定要删除医生 文医生（D260516004）吗？(y/n)：y

[成功] 医生删除成功！
```

12.4 查询医生

```
请输入选项：4

--- 医生查询 ---
1. 按ID精确查询
2. 按科室查询
3. 列出所有医生
请选择：1

请输入医生ID：D260514008

--- 查询结果 ---

[医生ID：D260514008]
  姓名：张医生      所属科室：皮肤科
  特长：擅长痤疮、痘印、湿疹、荨麻疹等常见皮肤病诊疗
```

```
请选择：2

请输入科室ID：K260514001

--- 科室 K260514001 的医生列表 ---

[医生ID：D260514008]
  姓名：张医生      所属科室：皮肤科
  特长：擅长痤疮、痘印、湿疹、荨麻疹等常见皮肤病诊疗

[医生ID：D260514009]
  姓名：李医生      所属科室：皮肤科
  特长：专攻过敏性皮炎、真菌感染、各类色斑与皮肤修护

[医生ID：D260514010]
  姓名：王医生      所属科室：皮肤科
  特长：精通皮肤科常见微创手术、皮肤医美及疤痕修复
```

```
请选择：3

--- 所有医生列表 ---

[医生ID：D260514008]
  姓名：张医生      所属科室：皮肤科
  特长：擅长痤疮、痘印、湿疹、荨麻疹等常见皮肤病诊疗

[医生ID：D260514009]
  姓名：李医生      所属科室：皮肤科
  特长：专攻过敏性皮炎、真菌感染、各类色斑与皮肤修护

[医生ID：D260514010]
  姓名：王医生      所属科室：皮肤科
  特长：精通皮肤科常见微创手术、皮肤医美及疤痕修复
```

13.床位管理

13.1 菜单

```
=====
                床位管理 [子菜单]
=====

1. 添加床位
2. 办理住院/出院 (修改状态)
3. 删除床位
4. 查询床位
0. 返回上级

=====
请输入选项:
```

13.2 添加床位

```
=====
科室ID: K260514007
科室名称: 心血管内科
科室医生数: 0
=====

请输入所属科室ID: K260514001

==== 输入床位信息 ====
1. 普通病房
2. 双人病房
3. VIP 病房
请选择病房类型: 1
[提示] 普通病房: 3人间, 标配医疗设备

[成功] 床位添加成功, 床位ID: B260516001
```

13.3 办理住院/出院 (修改状态)

住院

```
请输入选项: 2

请输入床位ID: B260516001

当前床位状态:

[床位ID: B260516001]
  所属科室: 皮肤科           病房类型: 普通
  状态: 空闲

--- 办理住院 ---
请输入患者ID: P260516001

[成功] 住院办理成功, 患者 P260516001 已入住床位 B260516001
```

出院

```
请输入选项: 2

请输入床位ID: B260516001

当前床位状态:

[床位ID: B260516001]
  所属科室: 皮肤科           病房类型: 普通
  状态: 占用   住院患者: P260516001  入住时间: 2026-05-16 20:25:34

--- 办理出院 ---
[确认] 确定要为患者 P260516001 办理出院吗? (y/n): y

[成功] 出院办理成功, 床位 B260516001 已释放
```


13.4 删除床位

```
请输入选项：3

请输入需要删除的床位ID（输入0取消）：B260516001

[确认] 确定要删除床位 B260516001 吗？(y/n)：y

[成功] 床位删除成功！
```

13.5 查询床位

菜单

```
请输入选项：4

--- 床位查询 ---
1. 按ID精确查询
2. 按科室查询
3. 按状态查询（空闲/占用）
4. 列出所有床位
请选择：1
```

按ID精确查询

```
请输入床位ID：B260516001

--- 查询结果 ---

[床位ID：B260516001]
  所属科室：皮肤科      病房类型：普通
  状态：空闲
```

按科室查询

```
请选择：2

请输入科室ID：K260514001

--- 科室 皮肤科 的床位列表 ---

[床位ID：B260514025]
  所属科室：皮肤科      病房类型：普通
  状态：空闲

[床位ID：B260514026]
  所属科室：皮肤科      病房类型：普通
  状态：空闲
```

按状态查询(空闲/占用)

```
请选择：3

请选择床位状态：0-空闲，1-占用：0

--- 空闲 床位列表 ---

[床位ID：B260514025]
  所属科室：皮肤科      病房类型：普通
  状态：空闲

[床位ID：B260514026]
  所属科室：皮肤科      病房类型：普通
  状态：空闲
```

列出所有床位

```
请选择：4

--- 所有床位列表 ---

[床位ID: B260514025]
  所属科室：皮肤科          病房类型：普通
  状态：空闲

[床位ID: B260514026]
  所属科室：皮肤科          病房类型：普通
  状态：空闲
```

14.床位统计查询

14.1 菜单

```
请输入选项：4

===== 统计查询 =====
1. 按科室统计床位使用情况
2. 全院床位统计
0. 返回上一级菜单
=====
请选择：
```

14.2 按科室统计床位使用情况

```
请选择：1

请输入科室ID: K260514001

===== 科室 皮肤科 床位统计 =====
总床位：7
已占用：0
空闲：7
占用率：0.0%
=====
```

14.3 全院床位统计

```
请选择：2

===== 全院床位统计 =====
总床位：34
已占用：0
空闲：34
占用率：0.0%
=====
```

15.排班管理

15.1 菜单

```
=====
                          排班管理
=====

1. 新增排班
2. 查看排班
3. 删除排班
4. 修改排班
0. 返回上级

=====
请输入选项：
```

15.2 新增排班

```
=====
科室ID: K260514007
科室名称: 心血管内科
科室医生数: 0
=====

请输入科室ID: K260514001

--- K260514001 的医生列表 ---
D260514008 | 张医生
D260514009 | 李医生
D260514010 | 王医生

请输入医生ID: D260514008
请输入排班日期 (YYYY-MM-DD): 2026-05-17
请输入时间段 (如 08:00-10:00): 8:00-10:00
请输入最大号源数 (该医生每日上限 30): 20

[成功] 排班添加成功!
排班ID: S260516002
医生ID: D260514008 | 日期: 2026-05-17 | 时段: 8:00-10:00 | 号源: 20
```

15.3 查看排班

菜单

```
请输入选项: 2

--- 查看排班 ---
1. 查看所有排班
2. 按医生查看
3. 按科室查看
4. 按日期查看
0. 取消
请选择:
```

查看所有排班

```
请选择: 1

[1] 排班ID: S260514059
    医生: 张医生 (D260514008) | 科室: 皮肤科
    日期: 2026-05-14 | 时段: 8:00-10:00
    号源: 0/10 | 状态: 可预约
[2] 排班ID: S260514060
    医生: 刘医生 (D260514011) | 科室: 呼吸科
    日期: 2026-05-14 | 时段: 8:00-10:00
    号源: 0/15 | 状态: 可预约
[3] 排班ID: S260516002
    医生: 张医生 (D260514008) | 科室: 皮肤科
    日期: 2026-05-17 | 时段: 8:00-10:00
    号源: 0/20 | 状态: 可预约

按回车键继续...|
```

按医生查看

```
请选择：2
请输入医生ID：D260514008

[1] 排班ID：S260514059
    医生：张医生（D260514008） | 科室：皮肤科
    日期：2026-05-14 | 时段：8:00-10:00
    号源：0/10 | 状态：可预约
[2] 排班ID：S260516002
    医生：张医生（D260514008） | 科室：皮肤科
    日期：2026-05-17 | 时段：8:00-10:00
    号源：0/20 | 状态：可预约

按回车键继续...
```

按科室查看

```
=====
科室ID：K260514007
科室名称：心血管内科
科室医生数：0
=====

请输入科室ID：K260514002

[1] 排班ID：S260514060
    医生：刘医生（D260514011） | 科室：呼吸科
    日期：2026-05-14 | 时段：8:00-10:00
    号源：0/15 | 状态：可预约

按回车键继续...
```

按日期查看

```
请选择：4
请输入日期（YYYY-MM-DD）：2026-05-17

[1] 排班ID：S260516002
    医生：张医生（D260514008） | 科室：皮肤科
    日期：2026-05-17 | 时段：8:00-10:00
    号源：0/20 | 状态：可预约

按回车键继续...
```

15.3 删除排班

```
请输入选项：3

--- 查看排班 ---
1. 查看所有排班
2. 按医生查看
3. 按科室查看
4. 按日期查看
0. 取消
请选择：4
请输入日期（YYYY-MM-DD）：2026-05-17

[1] 排班ID：S260516002
    医生：张医生（D260514008） | 科室：皮肤科
    日期：2026-05-17 | 时段：8:00-10:00
    号源：0/20 | 状态：可预约

按回车键继续...
请输入要删除的排班ID（输入0取消）：S260516002

确认删除排班 [S260516002] 2026-05-17 8:00-10:00 ? (y/n): y

[成功] 排班已删除。
```

15.4 修改排班

```
请输入选项：4

请输入要修改的排班ID：S260514059

=====
--- 修改排班 ---
排班ID：S260514059 | 科室：皮肤科 | 医生：张医生 | 日期：2026-05-14 | 时段：8:
00-10:00 | 已预约：0/10 | 状态：可用

1. 修改时段
2. 修改最大患者数
3. 切换可用状态
0. 保存并返回
请选择：0

[成功] 排班信息已保存。
```

16.药品管理与统计模块

```
=====
药品管理与统计模块
=====

1. 药品信息管理
2. 药品库存管理
3. 门诊发药
4. 全局查询统计
5. 一键数据备份
0. 返回主菜单

=====
请输入选项：
|
```

17.药品信息管理

17.1 添加药品

```
=====
科室ID：K260514007
科室名称：心血管内科
科室医生数：0
=====

请输入所属科室ID：K260514001
请输入药品通用名：丙酸氯倍他索
请输入药品商品名（可选回车）：恩肤霜
请输入药品别名（可选回车）：氯倍米松
请输入单价：10
请输入初始库存：100
请输入库存预警阈值（直接回车默认 20，即库存的 20%）：

[成功] 药品添加成功，药品ID：M260516003
```

17.2 修改药品信息

```
请输入选项：2

请输入需要修改的药品ID：M260516003

===== 修改药品信息 =====

[药品ID：M260516003]
通用名：丙酸氯倍他索      商品名：恩肤霜
别名：氯倍米松           所属科室：皮肤科
单价：10.00 元           库存：100 / 阈值：20 [[正常]充足]

=====
1. 修改通用名（当前：丙酸氯倍他索）
2. 修改商品名（当前：恩肤霜）
3. 修改别名（当前：氯倍米松）
4. 修改所属科室（当前：K260514001）
5. 修改单价（当前：10.00）
6. 修改库存（当前：100）
7. 修改预警阈值（当前：20）
0. 保存并返回

=====
请选择要修改的字段：0

[成功] 药品信息修改成功！
```

17.3 删除药品

```
请输入选项：3

请输入需要删除的药品ID（输入0取消）：M260516003

[错误] 该药品库存为 100，无法删除！
```

17.4 查询药品

菜单

```
请输入选项：4

--- 药品查询 ---
1. 按ID精确查询
2. 按关键词模糊查询（通用名/商品名/别名）
3. 按所属科室查询
4. 列出所有药品
请选择：
```

按ID精确查询

```
请选择：1

请输入药品ID：M260516003

--- 查询结果 ---

[药品ID：M260516003]
通用名：丙酸氯倍他索      商品名：恩肤霜
别名：氯倍米松           所属科室：皮肤科
单价：10.00 元           库存：100 / 阈值：20 [[正常]充足]
```

按关键词模糊查询(通用名/商品名/别名)

```
请选择：2

请输入查询关键词：米松

--- 包含 "米松" 的药品列表 ---

[药品ID: M260516003]
通用名：丙酸氯倍他索      商品名：恩肤霜
别名：氯倍米松           所属科室：皮肤科
单价：10.00 元           库存：100 / 阈值：20 [[正常]充足]
```

按所属科室查询

```
请选择：3

请输入科室ID：K260514001

--- 科室 K260514001 的药品列表 ---

[药品ID: M260516003]
通用名：丙酸氯倍他索      商品名：恩肤霜
别名：氯倍米松           所属科室：皮肤科
单价：10.00 元           库存：100 / 阈值：20 [[正常]充足]
```

列出所有药品

```
请选择：4

--- 所有药品列表 ---

[药品ID: M260516003]
通用名：丙酸氯倍他索      商品名：恩肤霜
别名：氯倍米松           所属科室：皮肤科
单价：10.00 元           库存：100 / 阈值：20 [[正常]充足]
```

18.药品库存管理

18.1 菜单

```
=====
                药品库存管理 [子菜单]
=====

1. 药品入库
2. 药品出库
3. 查看库存预警
0. 返回上级

=====
请输入选项： |
```

18.2 药品出库

```
请输入药品ID：M260516003

当前药品：丙酸氯倍他索 | 当前库存：100 | 预警阈值：20
请输入入库数量：100

[成功] 入库成功，当前库存：200
[操作记录] 2026-05-16 21:01:55 | 入库 丙酸氯倍他索 x100 | 经办人：管理员
```

18.3 药品出库

充足

```
请输入选项：2

请输入药品ID：M260516003

当前药品：丙酸氯倍他索 | 当前库存：200
请输入出库数量：10

[成功] 出库成功，当前库存：190
[操作记录] 2026-05-16 21:04:37 | 出库 丙酸氯倍他索 x10 | 经办人：管理员
```

不足

```
请输入选项：2

请输入药品ID：M260516003

当前药品：丙酸氯倍他索 | 当前库存：190
请输入出库数量：180

[成功] 出库成功，当前库存：10
[操作记录] 2026-05-16 21:10:44 | 出库 丙酸氯倍他索 x180 | 经办人：管理员

[警告] 当前库存（10）低于预警阈值（20），请及时补货！
```

18.4 查看库存预警

```
请输入选项：3

--- 库存预警药品列表 ---

[药品ID：M260516003]
通用名：丙酸氯倍他索      商品名：恩肤霜
别名：氯倍米松           所属科室：皮肤科
单价：10.00 元           库存：10 / 阈值：20  [[警告]不足]
```

19.门诊发药

```
=====
                门诊发药【子菜单】
=====

1. 门诊发药（模拟开药）
0. 返回上级

=====

请输入选项：1

--- 门诊发药 ---
请输入患者ID：P260517001
--- 请选择药品 ---
请输入药品ID：M260517001
请输入发药数量：1
请输入医生ID：D260514008

--- 费用明细 ---
药品：丙酸氯倍他索 x1
总费用：0.00 元
医保报销（70%）：0.00 元
患者自付：0.00 元
患者当前余额：97.00 元

[确认] 确认发药并扣费吗？(y/n)：y

[成功] 发药成功！
患者余额已扣除 0.00 元，当前余额：97.00 元
药品库存已扣除 1，当前库存：99
```

当前发药流程：管理员选择患者和药品，校验余额和库存后完成发药，自动生成处方记录。
本系统省略了"医生开处方 → 药房审核发药"的分步流程，将两步合并为管理员一键发药。

20.全局查询统计

20.1 菜单

```
=====
                        全局查询统计 【子菜单】
=====
1. 一站式全局查询
2. 管理员视角统计报表
0. 返回上级
=====
请输入选项： |
```

20.2 一站式全局查询

```
请输入选项：1

--- 一站式全局查询 ---
1. 查询患者 (住院/门诊)
2. 查询医生 (所属科室)
3. 查询科室 (医生数)
4. 查询床位 (状态)
5. 查询药品 (库存)
请选择：1

请输入ID: P260516001

--- 查询结果 ---
ID: P260516001 | 姓名：张三 | 年龄：24 | 性别：男 | 手机：15162482356 | 身份证：
110105200107081234 | 余额：100.00 | 医保：50% | 状态：未住院
```

20.3 管理员视角统计报表

```
=====
                        管理员视角统计报表
=====

[1] 基础数据概览
    科室数量：6
    医生数量：15
    患者数量：1
    床位数量：34
    药品数量：1
    医疗记录数量：0

[2] 床位使用情况
    总床位：34
    占用床位：0
    空闲床位：34
    全院床位使用率：0.0%

===== 药品库存预警 =====
当前库存低于预警阈值的药品数量：1 种
提示：请及时补货！
[导出] 报表已保存至 admin_stats_2026-05-16_21_22_46.txt
=====
```

21.一键数据备份

```
请输入选项：5

--- 一键数据备份 ---
正在备份所有数据...

[成功] 所有数据已备份！
患者数据：data/patient.txt
医生数据：data/doctor.txt
科室数据：data/dept.txt
床位数据：data/bed.txt
药品数据：data/drug.txt
记录数据：data/record.txt
排班数据：data/schedule.txt
预约数据：data/appointment.txt

提示：所有数据已自动保存到当前程序目录。
```

22.修改管理员密码

```
请输入选择：4

--- 修改管理员密码 ---
请输入当前密码：123456
请输入新密码：5525
请再次输入新密码：5525
[成功] 管理员密码已修改！
```

23.医生登录界面

```
请输入选择：2

=====
                医生登录系统
=====

请输入登录账号：zhang123
请输入密码：123456

=====
[登录成功] 医生 张医生 已登录。
```

24.医生菜单

```
=====
                医生工作站 [张医生]
=====

1. 查看我的患者
2. 管理医疗记录（新增/查看/修改状态）
3. 查看患者预约信息
4. 查看我的排班
5. 查看个人信息
6. 修改密码
0. 返回主菜单

=====
请输入选择：
```

25.查看我的患者

```
===== 查看我的患者 =====  
  
医生：张医生（科室ID: K260514001）  
  
--- 当前挂号患者列表 ---  
患者ID: P260516001 | 姓名：张三 | 状态：待就诊 | 挂号时间：2026-05-17 11:16:53  
  
--- 预约该医生的患者 ---  
预约ID: A260517003 | 患者：张三 | 日期：2026-05-17 8:00-10:00 | 状态：已预约
```

26.管理医疗记录（新增/查看/修改状态）

26.1 菜单

```
请输入选择：2  
  
===== 医疗记录管理（医生端） =====  
1. 查看患者医疗记录  
2. 新增诊断记录  
3. 新增处方记录  
4. 修改就诊状态  
0. 返回  
请选择：
```

26.2 查看患者医疗记录

```
请选择：1  
  
请输入患者ID: P260516001  
  
--- 患者的医疗记录 ---  
记录ID: R260516001 | 医生：张医生 | 类型：挂号 | 费用：5.00 | 时间：2026-05-16 21:37:00  
详情：普通挂号 - 费用：5.00  
记录ID: R260516002 | 医生：张医生 | 类型：挂号 | 费用：5.00 | 时间：2026-05-16 21:39:58  
详情：普通挂号 - 费用：5.00  
记录ID: R260517001 | 医生：张医生 | 类型：挂号 | 费用：5.00 | 时间：2026-05-17 11:16:53  
详情：普通挂号 - 费用：5.00  
  
按回车键继续...
```

26.3 新增诊断记录

```
请选择：2  
  
请输入患者ID: P260516001  
请输入诊断费用：10  
请输入诊断详情：基础检查  
  
[成功] 诊断记录已添加！
```

26.4 新增处方记录

```
请选择：3  
  
请输入患者ID: P260516001  
请输入处方费用：10  
请输入处方详情：开药  
  
[成功] 处方记录已添加！
```

26.5 修改就诊状态

```
请选择：4

请输入患者ID：P260516001

当前状态：待就诊
请选择新状态：
  1. 就诊中
  2. 已完成
  0. 取消
请选择：1

[成功] 就诊状态已更新！
```

27.查看患者预约信息

```
请输入选择：3

===== 查看我的预约 =====

医生：张医生

--- 预约列表 ---
  预约ID：A260517003 | 患者：张三 | 日期：2026-05-17 8:00-10:00 | 状态：已预约 |
  费用：10.00

按回车键继续...
```

28.查看我的排班

```
请输入选择：4

===== 我的排班 =====
  排班ID：S260514059 | 日期：2026-05-14 | 时间段：8:00-10:00 | 号源：0/10 | 状态
：可用
  排班ID：S260517002 | 日期：2026-05-17 | 时间段：8:00-10:00 | 号源：1/20 | 状态
：可用

按回车键继续...
```

29.查看个人信息

```
请输入选择：5

===== 个人信息 =====
  医生ID：D260514008
  姓名：张医生
  科室ID：K260514001
  擅长领域：擅长痤疮、痘印、湿疹、荨麻疹等常见皮肤病诊疗
  登录账号：zhang123
  每日挂号限额：30
  当前已挂号：1

按回车键继续...
```

30.修改密码

```
请输入选择：6

===== 修改密码 =====
请输入当前密码：123456
请输入新密码：5525
请再次输入新密码：5525
[成功] 密码已修改！
```

31.患者登录界面

```
请输入选择：3

===== 患者登录 =====
  请输入患者 ID 登录
  或输入 '1' 创建新患者
  输入 '0' 退出
请选择：
```

32.患者菜单

32.1 新增患者登录

```
请选择：1

--- 创建新患者 ---
请输入姓名：小美
请输入性别（男/女）：女
请输入手机号（11位手机号）：12345678912
请输入年龄（0-150）：11
请输入身份证号（18位）：431102200612200210
身份证出生日期计算年龄为19岁，与输入的11岁不符，请核对！

年龄和身份证号不一致，请重新输入。
请输入年龄（0-150）：19
请输入身份证号（18位）：431102200612200210
请设置6位访问密码（用于保护个人隐私，直接回车则不设置）：

[成功] 新患者 小美（ID：P260517001）已创建！
[成功] 新患者已创建，自动登录。
```

32.2 已有患者登录

```
请选择：P260516001

===== 患者自助服务 =====
  1. 普通挂号
  2. 预约挂号
  3. 查看我的挂号/预约记录
  4. 取消我的挂号/预约
  5. 查看我的医疗记录
  6. 自助充值
  0. 退出登录
请选择：
```

33.普通挂号

```
0. 退出登录
请选择：1

===== 普通挂号 =====

--- 选择科室 ---
可用的科室列表：
1. 皮肤科 (ID: K260514001)
2. 呼吸科 (ID: K260514002)
3. 骨科 (ID: K260514003)
4. 口腔科 (ID: K260514004)
5. 神经内科 (ID: K260514005)
6. 心血管内科 (ID: K260514007)
0. 取消
请选择科室编号：1

--- 皮肤科 的医生列表 ---
1. 张医生 | 擅长：擅长痤疮、痘印、湿疹、荨麻疹等常见皮肤病诊疗 | 剩余号源：29/30
2. 李医生 | 擅长：专攻过敏性皮炎、真菌感染、各类色斑与皮肤修护 | 剩余号源：25/25
3. 王医生 | 擅长：精通皮肤科常见微创手术、皮肤医美及疤痕修复 | 剩余号源：20/20
0. 取消
请选择医生编号：1

已选择医生：张医生

挂号费用：10.00 元 (医保报销 50%，实际支付：5.00 元)
确认挂号？(y/n)：y

【挂号成功】
患者：张三 (ID: P260516001)
医生ID: D260514008
支付金额：5.00 元
剩余余额：1981.00 元
挂号时间：2026-05-17 11:29:01
```

34.预约挂号

```
请选择：2

===== 预约挂号 =====

--- 选择科室 ---
可用的科室列表：
1. 皮肤科 (ID: K260514001)
2. 呼吸科 (ID: K260514002)
3. 骨科 (ID: K260514003)
4. 口腔科 (ID: K260514004)
5. 神经内科 (ID: K260514005)
6. 心血管内科 (ID: K260514007)
0. 取消
请选择科室编号：1

已选择科室：皮肤科

--- 选择预约时间 ---
1. 医生：张医生 | 日期：2026-05-17 | 时段：8:00-10:00 | 剩余：19/20
0. 取消
请选择排班编号：1

预约费用：20.00 元 (医保报销 50%，实际支付：10.00 元)
确认预约？(y/n)：y

【预约成功】
患者：张三 (ID: P260516001)
预约日期：2026-05-17 8:00-10:00
支付金额：10.00 元
剩余余额：1971.00 元
预约ID: A260517007
```

35.查看我的挂号/预约记录

请选择：3

--- 我的挂号/预约记录 ---

【当前挂号状态】

状态：待就诊

挂号时间：2026-05-17 11:29:01

挂号医生：张医生

【预约记录】

预约ID: A260517003 | 状态：已预约 | 时间：2026-05-17 11:18:23 | 费用：10.00

排班日期：2026-05-17, 时段：8:00-10:00

医生：张医生

预约ID: A260517007 | 状态：已预约 | 时间：2026-05-17 11:30:05 | 费用：10.00

排班日期：2026-05-17, 时段：8:00-10:00

医生：张医生

按回车键继续...

36.取消我的挂号/预约

36.1 菜单

请选择：4

--- 取消挂号/预约 ---

1. 取消当前现场挂号

2. 取消指定预约

0. 返回

请选择：

36.2 取消当前现场挂号

请选择：1

确认取消现场挂号？此操作不可恢复 (y/n): y

【警告】该患者已有诊断/处方记录，取消挂号不会自动删除。

【退款】已退还挂号费 5.00 元

【成功】已取消现场挂号，相关费用已退还。

36.3 取消指定预约

请输入要取消的预约ID: A260517003

确认取消预约 A260517003 ? (y/n): y

【成功】预约已取消，已退还 10.00 元。

37.查看我的医疗记录

请选择：5

--- 个人信息 ---

姓名：张三
年龄：24
性别：男
余额：1986.00

--- 医疗记录 ---

记录ID: R260516001 | 类型: 挂号 | 费用: 5.00 | 时间: 2026-05-16 21:37:00 (已取消)

详情: 普通挂号 - 费用: 5.00

记录ID: R260516002 | 类型: 挂号 | 费用: 5.00 | 时间: 2026-05-16 21:39:58 (已取消)

详情: 普通挂号 - 费用: 5.00

记录ID: R260517001 | 类型: 挂号 | 费用: 5.00 | 时间: 2026-05-17 11:16:53

详情: 普通挂号 - 费用: 5.00

记录ID: R260517004 | 类型: 诊断 | 费用: 10.00 | 时间: 2026-05-17 11:20:51

详情: 基础检查

记录ID: R260517005 | 类型: 处方 | 费用: 10.00 | 时间: 2026-05-17 11:21:46

详情: 开药

记录ID: R260517006 | 类型: 挂号 | 费用: 5.00 | 时间: 2026-05-17 11:29:01 (已取消)

详情: 普通挂号 - 费用: 5.00

记录ID: R260517008 | 类型: 挂号 | 费用: 5.00 | 时间: 2026-05-17 11:32:01 (已取消)

详情: 普通挂号 - 费用: 5.00

--- 费用汇总 ---

总记录数: 3
总费用: 25.00 元
医保预计报销: 12.50 元
预计自付: 12.50 元

按回车键继续...

38.自助充值

请选择：6

当前患者：张三 (ID: P260516001)
当前余额：1986.00 元

请输入充值金额 (正数, 单次不超过 100000.00 元, 输入 0 取消): 100

【成功】充值成功!
充值金额: 100.00 元
当前余额: 2086.00 元

十四、项目不足与改进

1. 链表查找性能瓶颈

系统所有查找操作基于 `FindNode()` 的 $O(n)$ 单向链表遍历。当前数据规模 (百级) 无影响, 但扩展到万级患者时, 按 ID 查找会产生明显延迟。

改进方向: 引入哈希索引或跳表结构, 将查找复杂度降至 $O(1)$ 或 $O(\log n)$ 。

2. 文本文件并发不安全

数据以纯文本文件持久化，无文件锁机制。多进程同时操作同一文件可能产生数据覆盖或读取不一致问题。

改进方向：引入文件锁（`flock` / `LockFileEx`）或迁移至嵌入式数据库（SQLite）。

3. 密码仅混淆不哈希

nibble-swap 是自逆变换，属于混淆（obfuscation）而非加密或哈希。任何有逆向工程能力的攻击者都可提取算法还原密码。

改进方向：正式环境中应使用 `bcrypt` / `Argon2` 等专业密码哈希算法。

4. 无完整事务回滚

发药、挂号等涉及多表联动的操作未实现 ACID 事务。虽然通过调整保存顺序（审计记录优先）降低了数据不一致风险，但中途崩溃仍可能产生部分写入。

改进方向：引入日志式提交（write-ahead log）或迁移至支持事务的数据库。

5. 缺少分页与搜索过滤

患者列表、医疗记录等大量数据一次性输出到终端，内容过长时直接刷屏。

改进方向：添加分页（`--More--` 暂停）、支持组合条件搜索（姓名 + 科室 + 日期范围）。

6. 控制台界面限制

系统基于 Windows 控制台，界面为字符菜单，无图形化交互。患者照片、药品图片、统计图表等无法直观展示。

改进方向：迁移至 Web 前端或接入 Qt 等 GUI 框架。

后记

从需求分析、系统设计、编码实现到测试验证，完整经历了一个小型软件项目的全流程。

1	项目回顾	
2		
3		
4	① 版本管理意识	Git 分支协作 · 提交粒度 · 代码审查
5	② 模块化设计	void* 泛型链表 · 策略模式 · 8 模块共用
6	③ 防御式编程	输入校验 · 原子写入 · 分精度计算
7	④ 测试驱动	98 例功能 · 44 项边界 · 12 项完整性
8		

① 版本管理意识

项目开发过程中，团队认识到版本控制的重要性。Git 的分支管理（feature branch / develop / main）、提交粒度控制、代码审查等机制，能有效提升多人协作效率。虽然在本次课程设计中未正式引入完整的 Git 工作流，但成员已在后续实践中开始使用。

② 模块化设计的价值

通过 `void*` 泛型链表和函数指针策略模式，8 个业务模块共用同一套数据管理骨架。新增实体只需提供 `format` + `parse` 两个函数即可接入，体现了"低耦合、高内聚"的设计思想。

③ 防御式编程

从 `getValidChoice` 的统一输入校验到原子写入、分精度金额计算，每一层都是在实际问题驱动下的防御性设计。先跑通、再加固的策略让开发节奏可控。

④ 测试驱动

98 个功能测试用例 + 44 项边界测试 + 12 项完整性验证，确保了核心业务逻辑在各种异常输入和边界条件下的正确性。

总的来说，本系统在课程设计范围内实现了稳定、可用的 HIS 核心功能，也为后续扩展和工程化留下了明确的方向。